

Thinking 8th

*An original adaptation of Leo Brodie's Thinking Forth, for the
8th language*

Contents

Preface	5
Getting 8th and Running Your First Program	6
What 8th is	6
Where to get it	7
Running a program	7
Trying a one-line expression	8
The interactive prompt	8
Where the official documentation lives	8
A Note on Notation	9
The stack	9
Printing text: <code>.</code> and <code>cr</code>	10
Reordering the stack: <code>dup</code> , <code>drop</code> , <code>swap</code>	10
Comments	11
Words, not functions	11
Defining a word: <code>:</code> and <code>;</code>	12
Namespaces: <code>n:</code> , <code>s:</code> , <code>a:</code> , and friends	12
Stack effects, and a shorthand for describing them	12
Variables: <code>var</code> , <code>var,</code> , <code>@</code> , <code>!</code>	13
Conditionals: <code>if</code> , <code>else</code> , <code>then</code>	14
Booleans	14
Chapter 1: The Philosophy of 8th	15
A Short History of Trying to Make Software Manageable	15
Words Are the Unit, Not Functions or Modules	16
Namespaces: A Lexicon You Don't Have to Invent	19
Hiding the Construction of a Data Structure	22
Is 8th a High-Level Language?	23
Performance and Portability	24
Summary	25
Chapter 2: Analysis	25
Iteration Beats Prediction	26
What Analysis Actually Produces	26
Sketching Interfaces in Words, Not Diagrams	27
Defining the Rules: From Prose to a Decision Table	28

Data Structures and the Limits of Generality	31
Summary	32
Chapter 3: Preliminary Design and Decomposition	32
Two Ways to Cut Up a Problem	32
Decomposition by Component: A Thermostat	33
A Change in Plan	36
Decomposition by Sequential Complexity, and Its One Wrinkle	38
The Limits of “Level” Thinking	40
Summary	41
Chapter 4: Detailed Design and Problem Solving	42
Getting Unstuck	42
Designing a Component	46
How 8th Wants to Be Written	46
Calculation, Data Structure, or Logic	48
Solving a Problem: Roman Numerals	49
Summary	53
Chapter 5: Elements of Style	53
Organizing Your Source	54
Formatting: Spacing and Indentation	55
Documenting Stack Effects, Precisely	55
Choosing Names	57
Summary	58
Chapter 6: Factoring	58
Factoring Out a Calculation	59
Don’t Push a Decision Down as a Flag	59
Factoring a Repeated Decision Into Data	61
Factoring Out Names: A Real Array, Not Parallel Variables	62
When to Factor	63
Factoring at Compile Time	64
Summary	65
Chapter 7: Taming the Stack	66
How Deep Is Too Deep?	66
Escaping the Stack: Word-Local Variables	67
An Auxiliary Stack for One Temporary Value	68
When a Save/Restore Turns Out to Be Bad Factoring	69
Sharing One Component for Two Purposes	70
Summary	72

Chapter 8: Bundling State, Redirecting Behavior	72
A Table of Related Values	72
Two Live States, One Set of Names	74
Redirecting a Word's Behavior	76
Summary	79
Chapter 9: Minimizing Control Structures	79
Guards and Dispatch, Not Nesting	80
Combining Conditions, and When Not To	81
Two Independent Things at Once	82
What You've Already Learned, Revisited	83
Calculating Instead of Deciding	84
Store the Value, Not a Flag About It	85
A Trick That Doesn't Travel	86
Summary	86
Epilogue: 8th's Effect on Thinking	87

An original adaptation of Leo Brodie’s *Thinking Forth*, for the 8th language.

Preface

Thinking 8th is an attempt to do for 8th what Leo Brodie’s *Thinking Forth* (1984) did for Forth: teach not the syntax of a language, but the way of thinking that makes it worth learning. Brodie’s book is not a manual. It is an argument about how to design software — when to factor, what to name, how to hide the parts of a program most likely to change — illustrated with a language that makes those decisions unusually visible. It is also, itself, a synthesis rather than one person’s solo argument: Brodie wrote and organized it, but its judgment is drawn as much from extensive interviews with Charles Moore, Forth’s inventor, and dozens of working Forth programmers as from Brodie’s own experience — a debt this book tries to preserve rather than flatten into “Brodie says,” wherever the distinction actually matters.

8th is a good language for the same kind of book, and a different one. It descends from Forth by way of Ron Aaron’s Reva Forth, and it keeps the things that made Brodie’s argument possible: words instead of functions, a data stack instead of named parameters, an interactive interpreter instead of a batch compiler. A reader who has never seen Forth will still recognize the shape of the ideas here. But 8th is not a 1980s Forth wearing new syntax. It has real namespaces instead of a naming convention. Its “variables” are one of several built-in container types, reference-counted and garbage collected, not raw addresses into memory you manage yourself. It runs the same source file on a desktop, a phone, and a server. Where those differences change the lesson, this book says so, instead of pretending 8th is Forth in disguise.

This is **not** a mechanical translation of *Thinking Forth*. Each chapter starts from what Brodie was actually trying to teach, separates that lesson from the Forth-specific mechanics he used to teach it, and then asks how the same lesson is naturally expressed in idiomatic 8th. Sometimes the answer is “almost exactly the same code, different words.” Sometimes it is “8th already has a real feature for the thing Forth programmers had to improvise.” Occasionally the honest answer is “this particular Forth technique doesn’t have a natural 8th analogue, and here’s why.” All three answers show up in this book.

Thinking Forth is available at [the Thinking Forth SourceForge project](#) under a Creative Commons Attribution-NonCommercial-ShareAlike 2.0 license, and a copy of its original LaTeX source is kept in this repository, under `thinking-forth-1.0/`, as reference material. This book is an original adaptation: original prose and original, independently verified 8th code, inspired by Brodie’s structure and spirit rather than copied from his text. It carries the same non-commercial, share-alike spirit forward.

Every code example in this book that can be run has been run, against the 8th distribution it was written against, and its actual output is shown alongside it. Where an example can’t reasonably be executed — because it needs a GUI, a network, hardware, or some other environment this book doesn’t assume — that is said plainly, rather than guessed at.

Read the next two short sections before Chapter 1: “Getting 8th and Running Your First Program,” which gets 8th installed and running on your own machine, and “A Note on Notation,” which explains a handful of things about how 8th source code is written in this book that will save you from misreading the very first examples.

Getting 8th and Running Your First Program

Everything from here on assumes you have 8th installed and running on your own machine, and that you’re trying the examples as you read them, not just reading them. This short section is the practical setup you need before that’s possible — where to get 8th, how to run a program, and how to try a single line of code without writing a whole file.

What 8th is

8th is a programming language and its own runtime: download one executable for your platform, and it both interprets 8th source files and gives you an interactive prompt to type code into directly. There’s no separate compiler, linker, or build system to install alongside it.

Where to get it

Go to <https://8th-dev.com/>. 8th is distributed by its author, Ron Aaron: complete the download/request form on the website, and you'll be sent a link to download the appropriate ZIP archive for your platform.

Unzip the distribution wherever you like. Inside it you'll find, among other things:

- the 8th executable for your platform;
- the official documentation;
- libraries;
- sample programs.

On Windows, for example, the 64-bit executable is found under `bin/win64/8th.exe`.

That's the only tool you need to follow this book — no package manager, no other software to install.

The exact download and licensing arrangements may change over time, so follow the current instructions on the 8th website rather than relying on this book for account or subscription details.

Running a program

Every code example in this book lives in its own file, one file per example, alongside the book's own materials. Running one is a single command: give the 8th executable the path to a `.8th` file, and it runs top to bottom, printing whatever the program prints.

For instance, the very first complete program this book shows you — [code/ch01/breakfast.8th](#), which appears again in Chapter 1 — runs like this:

```
8th code/ch01/breakfast.8th
```

```
cereal  
wash up
```

If the `8th` executable isn't on your system's `PATH`, use the full path to it instead — `bin/win64/8th.exe code/ch01/breakfast.8th`, or the equivalent for your

platform. Every example in this book was actually run this way before being written down, and the output shown is the real output, not a prediction.

Trying a one-line expression

You don't need a whole file to try something small. The `-e` option runs a single piece of code directly from the command line:

```
8th -e "3 4 n:+ . cr" -e bye
```

```
7
```

The trailing `-e bye` tells 8th to stop afterward — without it, 8th would carry on waiting for more input. This is a handy way to check a one-line question (“what does this word actually do?”) without creating a file for it.

The interactive prompt

Running `8th` with no file at all drops you into an interactive prompt: type an expression, press Enter, and 8th runs it immediately and shows you the result, the same way it would if that line were in a file. This is the fastest way to experiment once you're comfortable with the basics, and it's what “REPL” (read-eval-print loop) refers to elsewhere in this book and in 8th's own documentation.

One thing to know before your first launch: 8th's `help` and `apropos` words — which let you ask the interpreter about any built-in word by name — need a one-time setup step to work, run from inside the unzipped distribution:

```
8th bin/setup.8th
```

You don't need this to follow the book; every word this book uses is explained where it's introduced. It's worth doing anyway, the first time you install 8th, because it makes `help <word>` a fast way to check something this book hasn't covered yet.

Where the official documentation lives

Two places, both included in the distribution you downloaded:

- The `docs/md` folder contains the full manual, chapter by chapter, in plain Markdown.

- Once you’ve run `bin/setup.8th`, typing `help <word>` or `apropos <word>` at the interactive prompt searches that same manual for you, without leaving the terminal.

The same manual, along with a searchable index of every built-in word, is also published online at 8th-dev.com.

This book is deliberately more selective than the manual — it teaches the words you need, in the order you need them, and explains *why* 8th is shaped the way it is. The manual is where to go for the complete picture of a word this book hasn’t reached yet.

With 8th installed and a way to run code, you’re ready. Next: the handful of words and reading conventions this book relies on from the very first example onward.

A Note on Notation

This page teaches the handful of 8th words this book leans on constantly — enough to read every example in Chapter 1 without having to guess. You don’t need to have programmed in Forth, or in any other stack-based language, to follow it; nothing past this point assumes you have. If you *do* know Forth, a few short asides flag exactly where 8th behaves differently, but skip them freely — they’re bonus material, not prerequisites.

You don’t need to memorize any of this either. You need to have seen each piece once before it’s used in a real example, so you can read that example instead of puzzling over it.

The stack

8th passes data between words using a **data stack**: a last-in, first-out list of values that the whole running program shares. Typing a plain number pushes it onto the top of that stack. A word that needs input takes it from the top of the stack; a word that produces a result leaves that result on top for whatever comes next. There’s no other way values move around in 8th — no named parameters, no **return** statement.

Code runs left to right, one word at a time, exactly in the order you type it — no operator precedence, no evaluation order to puzzle out. Watch what happens when

three numbers are pushed and then printed one at a time with `.` (which prints whatever is currently on top of the stack, removing it as it does):

```
1 2 3
. cr
. cr
. cr
```

```
3
2
1
```

`1 2 3` pushes three values, in that order, so `3` ends up on *top*. `.` `cr` prints the top value and moves to a new line; doing that three times empties the stack from the top down, so the values come back out in the reverse of the order they went in. That’s what “last-in, first-out” means in practice, and it’s worth watching happen once before moving on.

Printing text: `.` and `cr`

`.` isn’t just for numbers — it prints whatever value is on top of the stack, of any kind, and removes it. `cr` prints a newline. Text in double quotes is a **string**, and, like a number, typing one pushes it onto the stack:

```
"hello" . cr
```

```
hello
```

`\n` inside a string is a newline character, so `"hello\n" .` prints the same thing as `"hello" . cr` — you’ll see both styles in this book.

Reordering the stack: `dup`, `drop`, `swap`

Three small words come up constantly because so much of 8th is about arranging values on the stack for the next word to use. `dup` copies the top value, so there are two of it; `drop` removes the top value entirely; `swap` exchanges the top two values:

```
5 dup . cr . cr      \ dup: copies 5, so two 5s print
1 2 drop . cr        \ drop: removes the 2, only 1 is left to
```

```
print
1 2 swap . cr . cr      \ swap: exchanges them, so 1 prints before 2
```

```
5
5
1
1
2
```

You’ll meet these again properly, with real problems to solve, later in the book — this is just so the names aren’t a surprise when they show up.

Comments

8th’s comment character is the backslash, `+\\+`, which runs to the end of the line (`--` does the same thing, in the SQL-comment style, and both can be used interchangeably). There is also a multi-line comment, `(* ... *)`, which nests. This book uses `+\\+` throughout, following the convention used in 8th’s own tutorials and sample code:

```
\\ this line explains the next one
1 2 swap . cr . cr    \\ prints 1, then 2
```

If you’ve used Forth: in most Forths, `( n1 n2 -- n3 )` is a comment — a stack-effect diagram that the compiler skips over. **In 8th, parentheses are not comments.** `( ... )` compiles the words inside it into an anonymous, callable block of code and leaves a reference to that block on the stack — a real, load-bearing feature (it’s how 8th builds things like loop bodies and callbacks), not something to be skipped over. A stray `( n1 n2 -- n3 )` in the middle of an 8th word will not be silently ignored; it will try to compile `n1`, `n2`, `--`, and `n3` as words, and fail. This one habit is worth actively unlearning if you’re coming from Forth, because 8th won’t warn you — it will just try to run `n1` as a word and complain that no such word exists.

Words, not functions

The unit of an 8th program — what another language would call a function, procedure, or method — is called a **word**. This book uses that term throughout, for a reason that will matter more once you start writing your own: a defining name

and an executable unit are the same thing in 8th, with no separate machinery of “calling” it. `.`, `cr`, and `dup` above are all words, exactly as much as anything you’re about to define yourself.

Defining a word: `:` and `;`

`:` starts a new definition; the name right after it is the word being defined; everything up to the matching `;` is that word’s body. Once defined, invoking the word by name runs its body from the top:

```
: greet    "hello, 8th!" . cr ;
```

```
greet
```

```
hello, 8th!
```

Read the definition as: “`define `greet`; its body is `print the string "hello, 8th!"`, followed by `print a newline`; done defining.” Then `greet`, typed on its own, runs that body.

Namespaces: `n:`, `s:`, `a:`, and friends

8th’s built-in words are grouped into **namespaces**, written as a short prefix before a colon — `n:` for words that work on numbers, `s:` for strings, `a:` for arrays, `m:` for maps, and so on. Addition, for instance, is `n:+`:

```
3 4 n:+ . cr
```

```
7
```

You’ll see this prefix constantly, and it’s not optional decoration — plain `+` by itself is not a word in 8th. Once you know the pattern, a new word like `n:-` or `n:*` needs no separate introduction: same namespace, different operation. Chapter 1 says more about why 8th is built this way.

Stack effects, and a shorthand for describing them

Consider a word that squares a number:

```
: square    dup n:* ;
```

In plain English: **square** expects to find one number sitting on the stack, and by the time it's done, it has left one number there too — the square of the one it started with. That's a word's **stack effect**: what it expects to find on the stack before it runs, and what it leaves there when it's done. Every word has one, whether or not anyone writes it down, and reading a definition — as you just did with **square** — is really the act of working out its stack effect.

Because a word's inputs and outputs never appear in its definition — no named parameters, nothing to read off the `:` line the way another language would show you — 8th programmers write that sentence down anyway, as a comment right after the word's name, in a compressed form:

```
: square    \ n -- n²
    dup n:* ;
```

`\ n -- n²` is nothing more than “starts with one number, ends with one number — specifically, its square” — squeezed down: whatever's before the `--` is what the word expects to find, whatever's after is what it leaves behind. Once you're used to reading it, the shorthand is faster than the sentence, which is the only reason to use it. This convention has a name, borrowed from Forth: a **stack-effect diagram**, or **SED**. A word that touches the stack only invisibly — reads or writes a variable, prints something, but doesn't itself take anything from or leave anything on the stack — is commented `\ --`.

Nothing enforces this comment or checks it against what the word actually does; it's purely for the reader. This book writes one only where it actually helps a definition make sense at a glance — not as decoration on every trivial word.

Variables: **var**, **var,**, **@**, **!**

A **variable** is a named container holding one value at a time. **var** declares one, initialized to **0**; **var,** declares one initialized to whatever is currently on top of the stack — so the value you want has to be pushed *before* **var,** runs:

```
0 var, count
```

Read this as: "``push 0`; then define **count** as a variable, initialized with that value."
count alone doesn't produce the value inside it — it produces a *reference* to the variable, which two more words act on: **@** (“fetch”) reads the value currently stored there, and **!** (“store”) writes a new one:

```
5 count !  
count @ . cr      \ prints 5
```

Read the first line as "`push 5; store it into **count**.`" Read the second as "`fetch the value in **count**; print it.`" The habit to unlearn, if you already know Forth's **VARIABLE**: **count** by itself is never the value in 8th — it's always the reference that @ and ! act on.

Conditionals: **if**, **else**, **then**

if expects a boolean on top of the stack (see below) and can only appear inside a word's definition, between : and ;. If the value is true, it runs the words up to the matching **else** (or **then**, if there's no **else**); if false, it skips to just after **else** and runs from there. Either way, execution continues after **then**:

```
: yes-or-no    \ flag --  
  if  
    "yes" .  
  else  
    "no" .  
  then  
  cr ;  
  
true yes-or-no  
false yes-or-no
```

```
yes  
no
```

Booleans

8th has real **true** and **false** words, not “any nonzero value.” This book uses them rather than raw -1/0 or 1/0, which is also the idiomatic style in 8th's own sample code.

That's the whole starting vocabulary. From here, every new word this book uses gets the same treatment — a short explanation and a small example — at the point where you first need it, and is assumed known afterward. On to the philosophy.

Chapter 1: The Philosophy of 8th

8th is a language, a runtime, and — like the language that inspired it — an implied argument about how software ought to be built. Nobody wrote that argument down as a manifesto. It shows up instead in the shape of the language itself: what’s easy to say, what’s hard to say, and what the language simply refuses to let you say. Before looking at 8th code, it’s worth asking where that shape came from, and what problem it was built to solve.

A Short History of Trying to Make Software Manageable

Early programs were bit patterns entered by hand — correct-or-not was the only question anyone asked. As machines and budgets grew, a second question appeared alongside correctness: could the program be *changed* without breaking it? Decades of language design have really been one long answer to that second question.

Assemblers gave instructions names instead of bit patterns. Macro assemblers gave repeated sequences of instructions names too. High-level languages broke the one-to-one link between what you typed and what the machine did, so $X = Y * (456/A) - 2$ could stand for a dozen machine instructions at once. Structured programming broke large problems into modules with one entrance and one exit, so a reader could reason about a piece of a program without holding the whole thing in their head.

Each of these was a real advance, and each one eventually ran into the same wall: a program decomposed by what it *does* — read the record, edit the record, write the record — falls apart the moment something about *how* it does it changes. Change the record’s layout and you’re back in all three modules at once, because all three modules knew that layout.

In 1972 David Parnas proposed a different criterion for drawing module boundaries: not sequence, not control flow, but **what is likely to change**. A module’s job, in this view, is to hide one such likely-to-change thing — a data layout, an algorithm, a piece of hardware — behind a small set of routines that the rest of the program uses instead of touching the thing directly. Get the boundary right, and a change stays inside the module that owns it. Barbara Liskov and Stephen Zilles gave the same idea a name a few years later: *data abstraction*. Their example was a stack: routines to push, pop, and test for empty, with the actual representation hidden behind them.

This is the idea 8th — like Forth before it — makes unusually natural to follow: not a design pattern you have to remember to apply, but close to the default shape of an 8th program. Build out of small, named words with data passed implicitly between them, and you are already most of the way toward decomposing by what might change, whether you set out to or not.

Words Are the Unit, Not Functions or Modules

Here is a complete, if small, 8th program:

```
true var, hurried

: hurried?  hurried @ ;
: cereal    "cereal\n" . ;
: eggs      "eggs and bacon\n" . ;
: clean     "wash up\n" . ;

: breakfast
  hurried? if cereal else eggs then clean ;

breakfast
```

Running it (see [code/ch01/breakfast.8th](#)) prints:

```
cereal
wash up
```

breakfast isn't a subroutine call away from **hurried?**, **cereal**, **eggs**, and **clean** — it's *built out of* them, the same way a sentence is built out of words rather than referring to them from a distance. 8th's own manual defines its basic unit of execution this way: "the same as a function, procedure, or routine in other languages" — but that undersells the difference in practice. There is no separate mechanism for defining a subroutine, declaring a variable, or writing the main program. A variable declaration (**true var, hurried**), a word that reads it (**hurried?**), and the word that ties them together (**breakfast**) are all just words, invoked the same way, by name. There is no **main()** that is structurally different from the functions it calls.

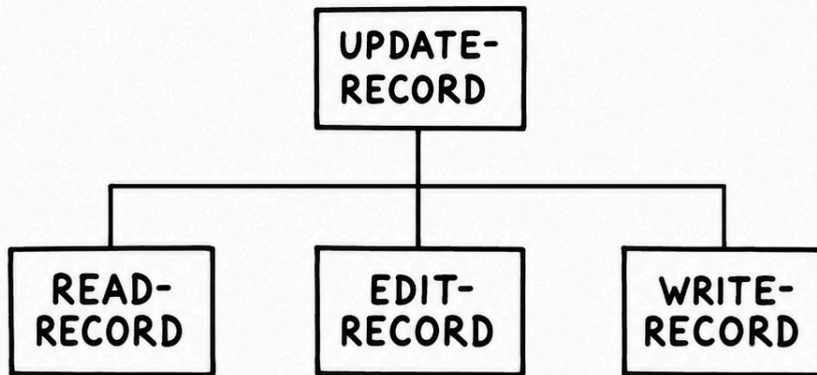
Two things about 8th make this possible — carried forward from two of Forth’s own founding design choices, which Brodie’s book takes as its starting point for everything that follows: calls are implicit, and so is data passing.

Calls are implicit. You don’t write `call cereal`; you write `cereal`. Every name 8th finds — a word, a variable, a constant — carries its own instructions for what to do when invoked. There is exactly one way to invoke anything: say its name.

Data passing is implicit. `hurried?` doesn’t take `hurried` as an argument in the conventional sense; it fetches the value and leaves it where the next word, `if`, expects to find it — on top of the data stack. `breakfast` never mentions the stack at all. It reads as a flat sequence of decisions: *are we hurried? if so, cereal, otherwise eggs; either way, clean up afterward*. The stack is the plumbing that makes this reads-like- a-sentence quality possible, and once you trust it, you stop thinking about it, the same way you stop thinking about which register a value sits in when you write `a + b` in almost any other language.

Because arguments travel on a shared stack instead of being named and declared, any word can be built out of any other words without either one knowing about the other’s internals. That’s what lets `breakfast` read as a single, flat idea instead of a nested tree of calls — and it’s also *exactly* Parnas’s information-hiding, arrived at as a side effect of how the language passes data around, rather than as a discipline layered on top.

Sequential/Hierarchical Design:



Design by Components:

: UPDATE-RECORD

RECORD READ EDIT RECORD WRITE ;

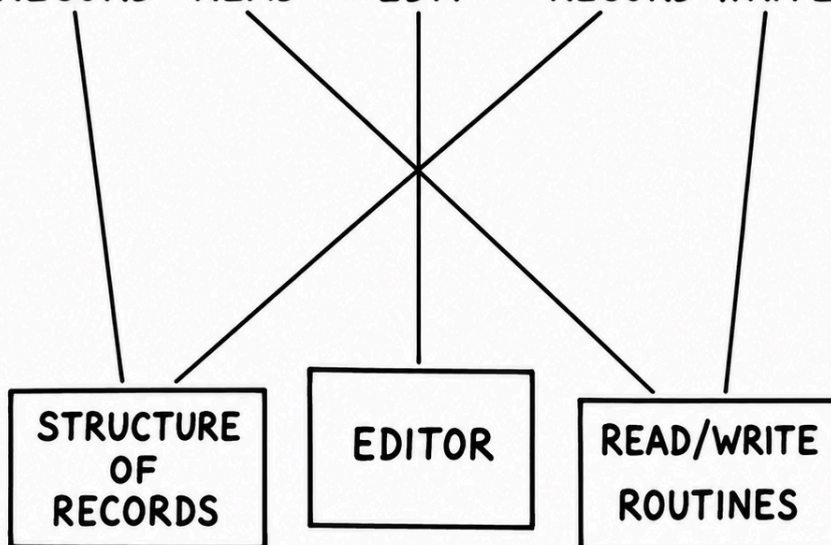


Figure 1: A hierarchical call tree, with one box calling three others below it, redrawn as the same functionality regrouped into three components that each hold their own data, functions, and structure.

Namespaces: A Lexicon You Don't Have to Invent

Brodie's book coins a term for a set of words that together hide one component's details from the rest of an application: a **lexicon** — “your interface with the component from the outside.”

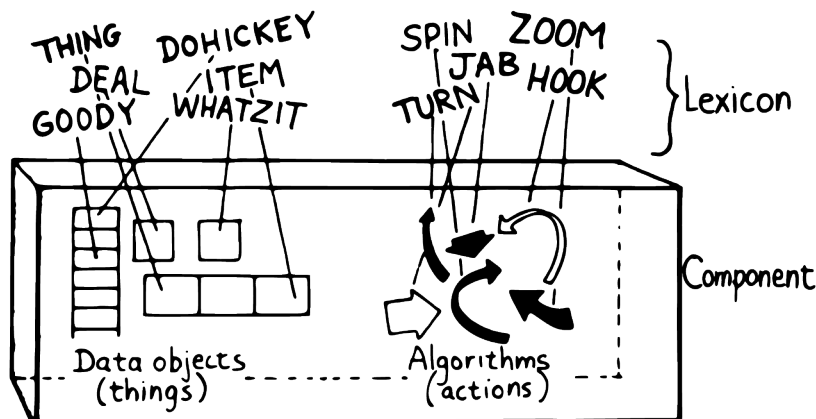


Figure 2: A component box, its internal structure and algorithms hidden inside, with a lexicon of named words bracketed around the outside as the only thing visible from beyond the boundary.

In classic Forth this is purely a design convention; nothing in the language enforces it or even knows it exists. A word belonging to a “stack” lexicon and a word belonging to a “queue” lexicon live in exactly the same flat dictionary, distinguished only by whatever naming discipline the programmer imposes.

8th takes the same idea and builds it into the language as a real feature: the **namespace**. Its own manual defines a namespace as “a vocabulary of (usually) related words” — which is Brodie’s definition of “lexicon,” independently arrived at and given a formal home in the dictionary rather than left to convention alone. Every built-in word that operates on numbers lives in the **n:** namespace — **n:** +, which “A Note on Notation” already showed you, along with **n:-**, **n:1-**, and others you’ll meet as they come up. Strings live in **s:**, arrays in **a:**, maps in **m:**, and so on. When you write your own component, you can give its words their own namespace prefix the same way, and 8th’s **with: / ;with** lets you temporarily bring a namespace’s words into scope unprefixed, for readability, without ever losing the boundary between components.

Worth being precise about what kind of boundary that is: a namespace organizes, it doesn’t lock. Nothing stops another component from calling **stock:internal-word** directly if it happens to know the name, any more than a Forth programmer

was ever physically stopped from reaching into someone else’s lexicon. The boundary is a naming discipline everyone agrees to respect, not an enforcement mechanism — which is exactly the distinction Brodie draws when he says information-hiding, done his way, is about protecting a design from *change*, not protecting one component from another the way a language with real access control would. (8th does have a genuine access-control word, **private**, but it’s scoped to a loaded library file, not to namespaces in general — a different tool, for a narrower job than a lexicon’s boundary.)

This is the one place in this chapter where “the natural 8th approach” is genuinely, structurally different from Forth’s, rather than a change of spelling: what Brodie has to argue readers *into* doing — decompose your program into small lexicons with clean boundaries — 8th’s own standard library already does, pervasively, as ordinary practice you’d have to work to avoid.

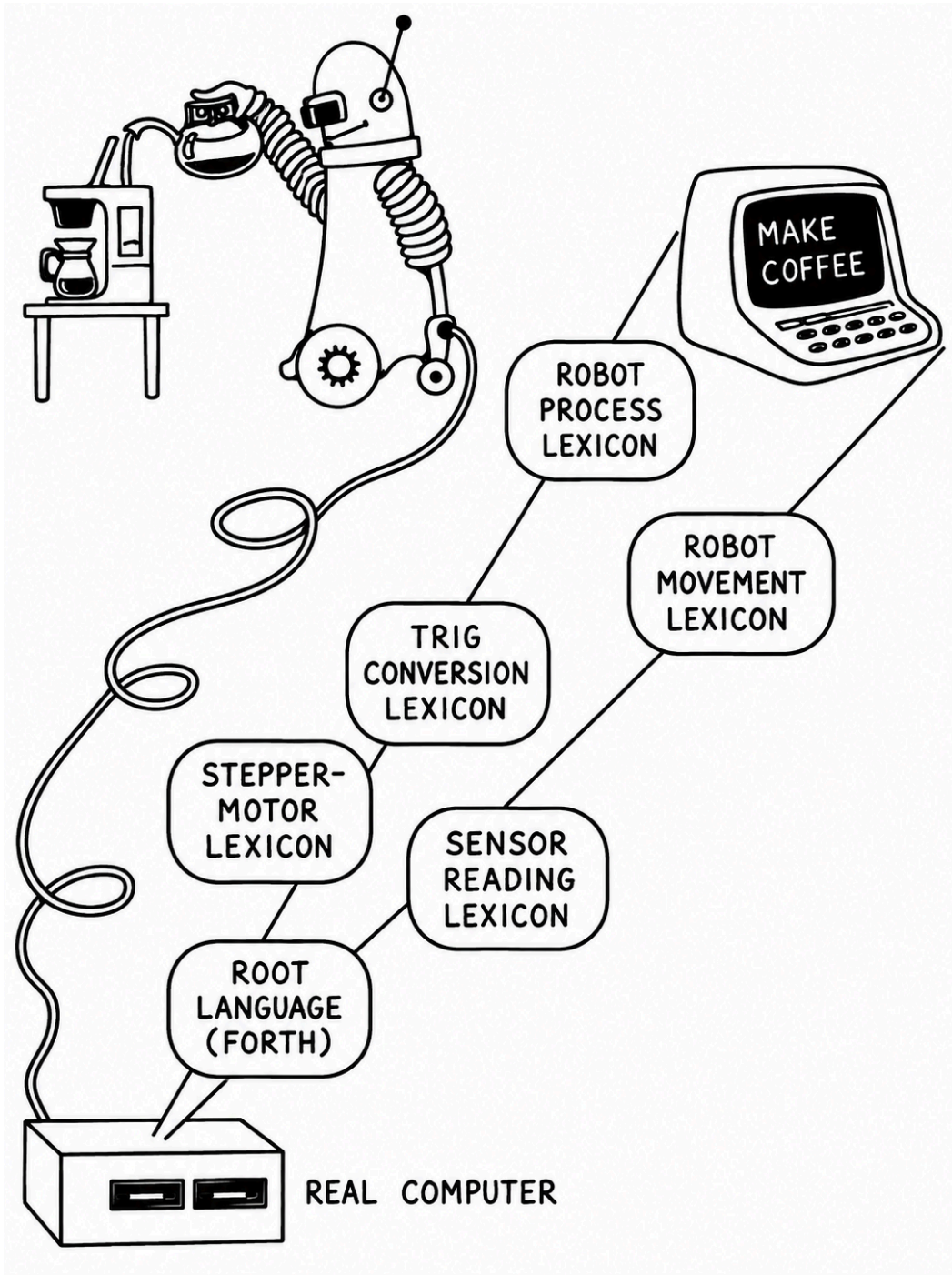


Figure 3: A whimsical robot, plugged into a coffee maker, built from a chain of lexicons stacked on top of a root Forth-like language – sensor reading, stepper-motor control, trigonometric conversion, robot movement, robot process – ending in a single command a person can type: MAKE COFFEE.

A real robotics application built this way genuinely could end in a single word that reads as plainly as that picture suggests — every layer beneath it a lexicon hiding the one below, until “make coffee” is a complete sentence a machine can act on.

Hiding the Construction of a Data Structure

Brodie’s central example of information-hiding is a variable called **APPLES**, used to tally apples, that later needs to become two variables — one for red apples, one for green — without changing a single line of code that already uses **APPLES**. The trick works in 8th exactly the way it works in Forth, for the same underlying reason: a variable’s *name* and the *value it produces when read* are two different things, so the second can be redefined without disturbing the first.

Start with a plain variable. One new word appears below: **n:+**! takes a number and a variable reference, and adds the number to whatever the variable currently holds, in place — a shorthand for fetching, adding, and storing back, in one step:

```
0 var, apples

20 apples !
apples @ . cr           \ => 20

1 apples n:+!
apples @ . cr           \ => 21
```

Now suppose, after code elsewhere already depends on **apples**, you discover you need two tallies — red and green — selected by which color is “current”:

```
0 var, color           \ which color is "current"?
0 var, reds
0 var, greens

: red    reds color ! ;
: green  greens color ! ;

: apples color @ ;
```

apples is no longer a variable at all — it’s a *word* that returns whichever variable is currently selected. But because **apples @**, **apples !**, and **apples n:+**! all still work exactly as before, nothing that used **apples** needs to change:

```

red
20 apples !
apples @ . cr                \ => 20

green
5 apples !
apples @ . cr                \ => 5

1 apples n:+!
apples @ . cr                \ => 6

red
apples @ . cr                \ => 20   (still there, untouched)

```

This is [code/ch01/apples.8th](#), run and verified against the actual output shown above. One detail worth knowing about, because it’s genuinely useful rather than merely cosmetic: when the file redefines **apples** from a variable into a word, 8th prints a warning to the console —

```
Redefining: user:apples
```

— and then proceeds. This is 8th telling you, out loud, exactly the thing this example is about: you have replaced an existing name with a new meaning. In a large program, that warning is a safety net; here, it’s confirmation that the trick worked.

What made this possible is the same pair of features from the last section, applied to *nouns* instead of *verbs*: **apples** is a word, so calling it is implicit; and it produces a reference on the stack rather than a name in the source text, so **@**, **!**, and **n:+!** don’t need to know or care whether that reference came from a plain variable or from three lines of logic. 8th, like Forth, doesn’t force you to distinguish between “a thing” and “an action that produces a thing” — a word can play either role, and nothing about how you invoke it gives away which one it’s playing today.

Is 8th a High-Level Language?

By the standard measure — does it hide the correspondence between source code and machine operations — 8th is unambiguously high-level; it runs on top of 8th’s own interpreter engine, not on a specific processor’s instruction set, and the same source file runs unmodified on a desktop, a phone, or a server. By another standard

measure — strict syntax checking that catches your mistakes before you run the program — 8th, again like Forth, does almost none. Write **apples red** where you meant **red apples** and 8th won't stop you; it will simply do what you said, which is not what you meant.

The trade this makes is the same one Brodie described in 1984: in exchange for very little static protection, you get a language with no fixed grammar to fight. Adding a new *kind* of word — a new control-flow construct, a new way of defining something — is not a special, harder kind of programming in 8th; it's what the words **if**, **var**, and **:** themselves are, examples of an extension mechanism available to you as much as it was available to whoever wrote those particular words. There is no wall between “the language” and “code you wrote.”

8th also leans further into interactivity than most languages that came after Forth. There's no edit-compile-link-test cycle; you type a phrase and the interpreter answers immediately, whether that phrase is a whole program or three words you're checking the behavior of. This turns “design” and “test” into the same activity rather than sequential phases — you can write the outermost, most abstract word of an application first and give its supporting words trivial, placeholder definitions, running the whole thing end-to-end from day one, then replace the placeholders one at a time as the real implementation gets built underneath them. This isn't a workaround for the absence of a proper design phase; it *is* a design method, and it depends on exactly the same word-at-a-time, implicit-calls, implicit-data-passing properties this chapter has been describing.

Performance and Portability

Brodie spent several pages of the original chapter arguing that Forth, despite its unusual appearance, was competitive with assembly language in size and speed, thanks to a compilation technique called threaded code. That argument doesn't carry over to 8th unchanged, and it would be dishonest to pretend otherwise: by 8th's own documentation, compiling a word packs what it needs into an internal code cache rather than emitting native machine instructions (an earlier version of 8th could generate native code, but that path was dropped because of restrictions on the iOS platform) — and, by explicit design choice, 8th performs no optimization on your code at all, apart from tail-call elimination. The reasoning given is not “we haven't gotten to it yet”; it's that an optimizer can silently change a program's behavior, and that the most effective optimization available is still a programmer choosing a better algorithm.

What 8th trades raw execution speed for is something Forth in 1984 could not offer: the same source file, unmodified, targets desktop operating systems, mobile platforms, and embedded devices from one implementation. Where Brodie’s Forth achieved portability across hardware by being small enough to reimplement on each new target, 8th achieves it by being one implementation that already runs everywhere. Different eras, different scarce resource, same underlying value — write the logic once, in a language that gets out of the way.

Summary

Strip away the unfamiliar punctuation, and 8th is making the same wager Forth made: that a program built entirely out of small, named, freely composable words — with data flowing between them implicitly, on a stack, rather than declared and passed by hand — makes Parnas’s kind of change-driven decomposition unusually natural to fall into, rather than something you have to impose on top of subroutines, modules, or objects with explicit interfaces. Where Forth left the discipline of grouping those words into coherent components (“lexicons,” in Brodie’s term) up to the programmer, 8th builds a formal version of the same idea into the language as namespaces. Where Forth achieved portability by being small and easy to port, 8th achieves it by being one implementation that already runs everywhere you’re likely to want to deploy.

None of this is free. You give up static type-checking, a fixed grammar to lean on, and — compared to hand-tuned native code — raw speed. What you get in exchange is a language with almost nothing between your intent and the running program: no ceremony for declaring a subroutine, no boilerplate for passing arguments, no separate compile step standing between a change and seeing it run. The rest of this book is about what that trade lets you build, and how to build it well.

Chapter 2: Analysis

Nobody agrees on how many phases software development has. Brodie counted nine, from discovering requirements down to maintenance, and then spent the rest of his second chapter demonstrating that Forth programmers routinely ignore the ordering. The same is true of 8th, for the same underlying reason: a language where you can write one word, test it in the running system, and move on doesn’t force

analysis, design, and implementation into separate, sequential phases. It lets them interleave.

That’s a genuine advantage, not an excuse to skip thinking. This chapter is about what to think about before — and while — you write 8th code that matters.

Iteration Beats Prediction

Brodie interviewed working Forth programmers throughout the 1980s, and a pattern emerged that had nothing to do with Forth specifically: the programmers who did the best work were not the ones who planned the most, or the ones who planned the least, but the ones who treated their early code as a question rather than an answer. Build the smallest thing that tells you something true about the problem, show it to the people who’ll actually use it, and let what you learn reshape the next version. Planning still matters — nobody plans nothing on a project worth doing — but it has diminishing, and eventually negative, returns. Past a certain point, planning is a way of avoiding the discovery that your plan was wrong.

8th’s whole design leans into this. There’s no build step standing between “I changed the code” and “I can see what it does now.” A word you’re unsure about can be defined with a placeholder body — print its own name, return a fixed value, do nothing — and wired into the rest of the program immediately, so that the shape of the whole system is testable from the very first day, long before every part of it is real. Later chapters return to this technique in more depth; this chapter is about how to decide *what* those words should be before you write any of them.

What Analysis Actually Produces

Whatever you call the phase, analysis has to answer three questions before serious implementation starts:

1. What does the system actually need to do, and what constraints (time, memory, an existing device it has to talk to, a deadline) bound the answer?
2. What’s the simplest model of a solution that satisfies those needs?
3. Roughly, what will it cost — in time, and in the resources the target system actually has — to build that model?

The rest of this chapter is about the second question, because it’s the one a language can actually help with.

Sketching Interfaces in Words, Not Diagrams

A common technique for the first question is the data-flow diagram: circles for operations, arrows for the data moving between them.

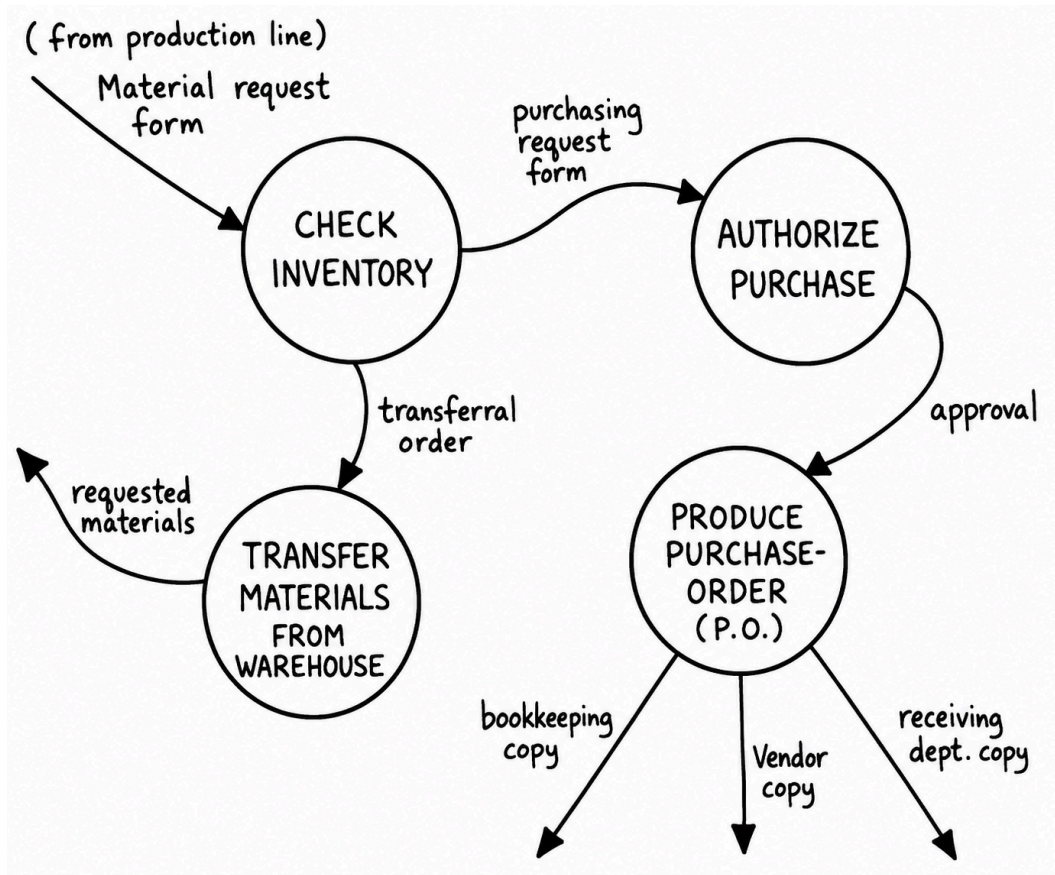


Figure 4: A data-flow diagram: circled operations CHECK INVENTORY, AUTHORIZE PURCHASE, and PRODUCE PURCHASE-ORDER, connected by labeled arrows for the forms and approvals passing between them, with a TRANSFER MATERIALS FROM WAREHOUSE step branching off to the side.

It's a useful tool for explaining a design to someone who doesn't read code. But if the person you're explaining it to *does* read code, a word-based language can often skip the diagram and go straight to something almost as readable, and far more useful, because you can run it. One new word appears below: **not** takes a boolean off the stack and pushes the opposite one back.

```

false var, garage-full?

: space-available?  garage-full? @ not ;
: let-in    "Welcome -- take a ticket.\n" . ;
: turn-away "Sorry, we're full.\n" . ;

: admit-car \ --
  space-available? if let-in else turn-away then ;

```

This is a complete, if trivial, working sketch of a parking garage’s entry policy — not pseudocode dressed up to look like 8th, but real, runnable 8th in which the interesting decision (**space-available?**) is separated from the actions it chooses between (**let-in**, **turn-away**), and the top-level word (**admit-car**) reads as a flat sentence describing the policy. Nothing here commits you to how a car’s arrival is actually detected, how a ticket is actually printed, or how **garage-full?** actually gets set — those are implementation details you can fill in underneath this sketch, one word at a time, without changing **admit-car** at all. Run it and it behaves exactly as the words suggest:

```

admit-car
true garage-full? !
admit-car

```

prints

```

Welcome -- take a ticket.
Sorry, we're full.

```

That’s the whole point. A design sketch you can execute catches a misunderstanding immediately — try the phrase, watch what happens — instead of after the diagram has been approved and handed off for implementation.

Defining the Rules: From Prose to a Decision Table

Interfaces are usually the bulk of an analysis. Occasionally, though, an application has genuine *rules* — logic complicated enough that a sentence of English doesn’t capture it safely. Suppose our garage’s exit fee depends on when you parked (weekday daytime, weekday evening, or weekend), how many hours you stayed, and whether you used valet service. Written as a paragraph, the rate schedule reads

about as well as a tax form: “During weekday daytime hours the charge is \$4.00 for the first hour and \$2.00 for each additional hour... on weekday evenings, \$2.00 for the first hour and \$1.00 for each additional hour... on weekends, a flat \$1.00 per hour... valet service adds a flat \$5.00 regardless of the hour or day.” It’s not wrong, it’s just hard to check for gaps or contradictions by eye.

Turning the same rule into nested conditionals doesn’t help much — you’d get a wall of **if/else** three or four levels deep, repeating the “additional hour” logic inside every branch of the “which tier” decision, which obscures the one fact that actually matters for simplifying the problem: **the per-hour rates depend only on which tier you’re in, and nothing else.** A **decision table** — tier along one axis, first-hour and additional-hour rates along the other — makes that fact visible at a glance, in a way prose and nested conditionals both bury.

	first hour	additional hour
weekday day	\$4.00	\$2.00
weekday evening	\$2.00	\$1.00
weekend	\$1.00	\$1.00

Once the rule is a table, 8th lets you implement it as an actual table, looked up by index, rather than as a chain of comparisons pretending to be one. That takes three pieces of new vocabulary, each small.

constant is like **var**, except the value can never change afterward, and reading it back needs no **@** — the name alone produces the value:

```
3 constant three
three . cr      \ prints 3
```

Square brackets, with commas between items, build an **array** — an ordered list, indexed from **0**:

```
[ "zero" , "one" , "two" ] constant names
```

And **caseof** looks a value up by position: give it an array and an index, and it returns whatever is stored at that position (or, for a map, give it a string key instead of a number). If what’s stored there happens to be a word, **caseof** calls it and returns the result instead of the word itself:

```
names 0 caseof . cr    \ prints zero
names 1 caseof . cr    \ prints one
```

Read as “look this up,” a decision table and a **caseof** array are the same idea — put the whole table in an array, in tier order, and let **caseof** do the looking up:

```
0 constant DAY
1 constant EVENING
2 constant WEEKEND

0 var, tier
false var, valet?

: set-day      DAY tier ! ;
: set-evening  EVENING tier ! ;
: set-weekend  WEEKEND tier ! ;
: valet-on     true valet? ! ;

[ 400 , 200 , 100 ] constant first-hour-rates  \ cents
[ 200 , 100 , 100 ] constant addl-hour-rates   \ cents

: first-hour-rate  \ -- cents
  first-hour-rates tier @ caseof ;

: addl-hour-rate   \ -- cents
  addl-hour-rates tier @ caseof ;
```

The valet surcharge isn’t part of the table at all — it doesn’t depend on the tier or the hour, so tying it to either would be exactly the kind of false coupling Chapter 1 warned about. It’s its own small word:

```
: valet-surcharge  \ cents -- cents
  valet? @ if 500 n:+ then ;
```

And the whole fee is a composition of these three pieces, matching the shape of the table instead of hiding it:

```
: parking-fee  \ hours -- cents
  1 n:-  addl-hour-rate n:*
```

```
first-hour-rate n:+  
valet-surcharge ;
```

This is [code/ch02/parking-fee.8th](#), executed and checked against hand-calculated expectations:

set-day	3	parking-fee . cr	\ => 800	(400 + 2*200)
set-evening	3	parking-fee . cr	\ => 400	(200 + 2*100)
set-weekend	5	parking-fee . cr	\ => 500	(100 + 4*100)
valet-on				
set-day	1	parking-fee . cr	\ => 900	(400 + 0 + 500)

Notice what happened to the + in the table: in the paragraph-of-prose version it appeared nine times, once per cell. In the factored version it appears exactly twice — once combining the two rate components, once adding the surcharge — because the table stopped being nine separate facts and became one small idea (a per-tier rate) applied uniformly. That collapse from nine cases to one idea *is* the analysis. The 8th code is just where the analysis stops being deniable: if the factoring is wrong, **parking-fee** gives you the wrong number, immediately, rather than a diagram nobody double-checked.

Data Structures and the Limits of Generality

Sometimes analysis has a third job: deciding what to remember, not just what to do. A parking garage that only ever admits and charges one car at a time doesn't need much of a data structure. One that needs to know which of its two hundred spaces are occupied, by which ticket number, since what time, is a different problem — and *that* decision (one record per space? one growing log of entries and exits?) belongs in analysis, before any code commits you to it, because it's exactly the kind of thing Chapter 1 called “likely to change”: today it's spaces in a garage, next year it might be spaces in three garages, or hourly and monthly permit-holders sharing the same lot.

The temptation, faced with that uncertainty, is to generalize: build a data structure that could handle any garage configuration anyone might ever want. Resist it — Charles Moore's own rule of thumb, drawn out at length in Brodie's interviews with him, is blunt about exactly this temptation: generality usually just means complexity, and a solution should stay sized to the problem it actually has, not the one it might face someday. A solution sized for problems you don't have yet

is usually harder to understand, harder to verify, and — because generalization multiplies the number of cases that interact with each other — no easier to change than one sized for the problem you actually have, built so that the parts likely to change are hidden behind a small lexicon of words, the way **tier**, **first-hour-rate**, and **valet?** hide the rate structure above. Simple and changeable beats general, almost every time.

Summary

Analysis, in 8th as in Forth, isn't a document you produce before coding starts — it's the activity of finding the smallest accurate model of the problem, however long that takes, and 8th's short feedback loop makes it cheap to test that model as you refine it rather than only after it's finished. Interfaces are usually best expressed directly as words with placeholder bodies, executable from day one. Genuinely complicated rules are best captured as decision tables, and 8th's **caseof** lets a decision table survive into the running program as an actual table instead of dissolving into a maze of conditionals. And whatever data structures the problem needs should be sized to the problem you have, not the one you can imagine — because the parts you can't predict are exactly the parts you'll want to have hidden behind a word, ready to change.

Chapter 3: Preliminary Design and Decomposition

Analysis tells you what a program has to do. Preliminary design is the next step: deciding what pieces it should be built from. Get this step right and implementation is a series of small, well-defined problems. Get it wrong, and you spend implementation discovering the right decomposition anyway — the hard way, by fighting code that doesn't want to bend the way the requirements just bent.

Brodie describes two ways to divide a program into pieces. This chapter works through both, with one running example, in 8th.

Two Ways to Cut Up a Problem

The first way is **decomposition by component**: group words by the thing they know about — a data structure, a device, a rule — regardless of when in the

program’s execution that knowledge gets used. This is the approach Chapter 1 already argued for: components as Parnas-style likely-to-change boundaries, given a name (a namespace, in 8th) and a small set of words as their public face.

The second is **decomposition by sequential complexity**: since a word has to exist before anything can call it, a program built word by word tends to arrange itself from simplest to most capable, the way a textbook moves from basic facts to advanced ones. This isn’t really a design choice — it’s a consequence of writing in a word-based language at all — but it has implications worth understanding, including one genuine wrinkle: sometimes a foundational word needs to call something that, by the “simplest first” ordering, hasn’t been written yet.

Both approaches show up in the example below.

Decomposition by Component: A Thermostat

Suppose the problem is a thermostat: read a temperature, decide whether to heat, cool, or do nothing, and let a person override the decision manually. Before writing any of that, ask what a *component* boundary looks like here. One candidate jumps out immediately: whatever “the current mode” is and how it gets changed is exactly the kind of thing likely to grow more rules later (a minimum-run timer, a “don’t switch modes twice in five minutes” guard) — so it belongs behind its own small set of words, not scattered through whatever code happens to decide when to heat or cool.

```
0 constant IDLE
1 constant HEATING
2 constant COOLING

IDLE var, mode

: mode@      \ -- mode
  mode @ ;

: set-mode    \ new-mode --
  dup mode@ n:= not if
    dup mode !
  else
    drop
  then ;
```

`set-mode` isn't just "store a number in a variable" dressed up. Look at what it's already doing: it's the one place in the whole program that ever sees *both* the old mode and the new one at the same moment, because it's the one place any code goes through to change the mode at all. Nothing outside this handful of words ever touches the `mode` variable directly — not even to read it, which is what `mode@` is for. That discipline is worth naming, because it's what Chapter 1 called an interface component: whatever data two or more other parts of the program need to share should live behind its own words, not be reached into directly, precisely so that a rule about *how* it's shared (like "only change it if it's actually different") has exactly one place to live.

The naive alternative, avoided here, is each side of a boundary keeping its own copy of whatever they share, meeting only at a narrow, easy-to-get- wrong junction:

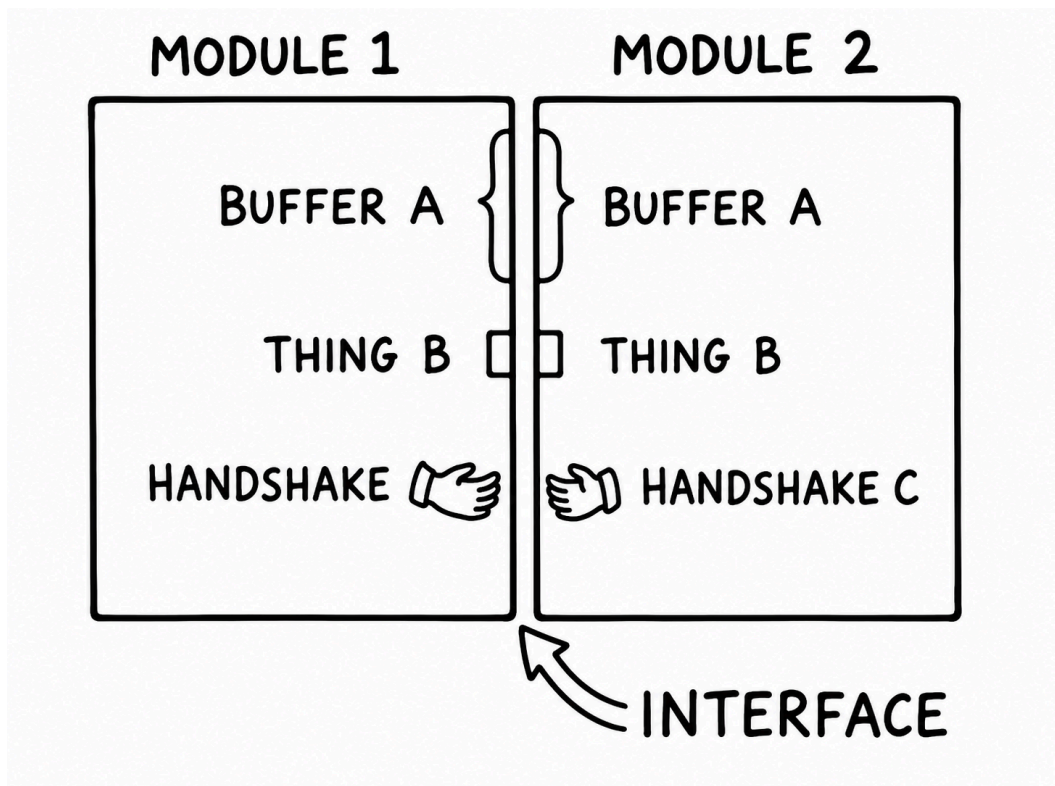


Figure 5: Two modules meeting at a jagged boundary, each maintaining its own separate copy of *BUFFER A* and *THING B*, joined only by a handshake at the seam.

against a design where the shared data lives once, in a component both sides call into rather than duplicate:

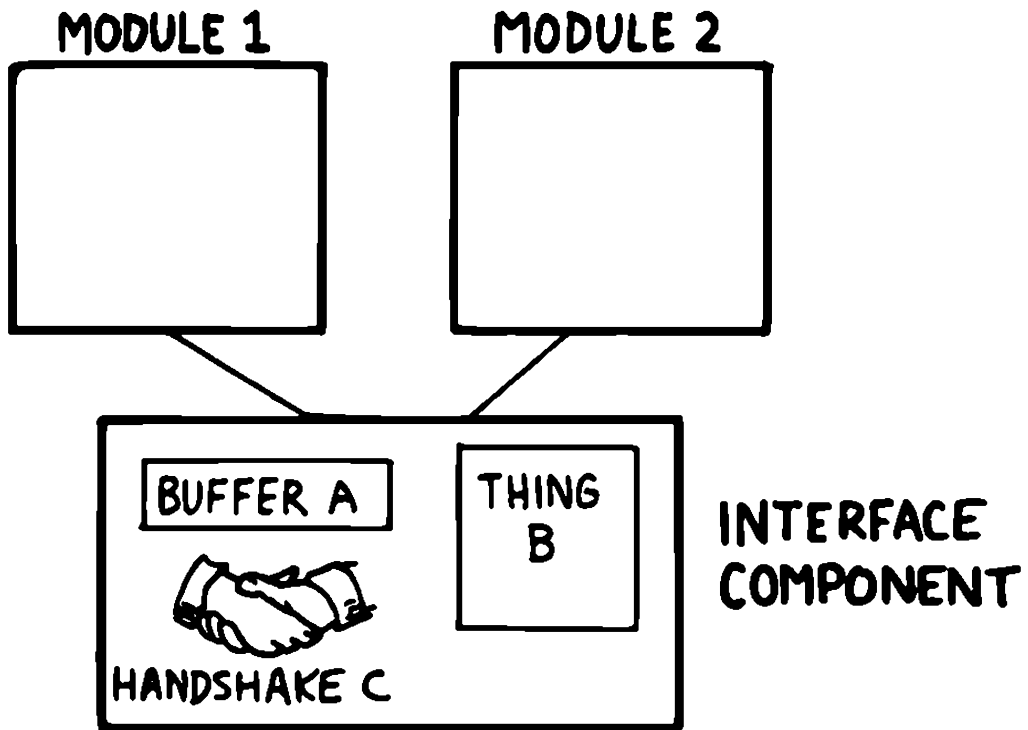


Figure 6: The same two modules, now empty, each with a single line down to one shared box labeled *INTERFACE COMPONENT* holding one copy of *BUFFER A*, *THING B*, and the handshake between them.

`mode/mode@/set-mode` is that second picture: one shared component, called into from both the automatic and the manual side, rather than two copies of “the current mode” that could quietly drift apart.

One more discipline belongs here, easy to miss until it’s violated: whatever crosses an interface should be expressed in terms the rest of the program can use without knowing how it was produced. `read-temp` already does this correctly, later in this chapter — it will return degrees, not a raw sensor voltage or an ADC reading — which is exactly why `decide-mode` can compare against `68` and `76` without caring whether the sensor underneath is a thermistor or a thermocouple. Get this backward, and the mistake is easy to make invisibly: let two components quietly share one raw variable that was only ever meant for one of them, and a change to that variable breaks the other in a spot neither one was looking at. `mode` is protected from exactly this failure by the same discipline that protects it from duplication — nothing outside `set-mode` ever touches the variable directly, so

there's no raw shared state left exposed for a second component to lean on by accident.

Now the two things that actually want to change the mode. The automatic decision uses two new comparisons: `n:<` and `n:>` both take two numbers and push a boolean, in the order you'd read them aloud — `a b n:<` asks "`a` is `a` less than `b`", not the other way around:

```
: decide-mode \ degrees -- new-mode
  dup 68 n:< if
    drop HEATING
  else
    76 n:> if COOLING else IDLE then
  then ;
```

And a person overriding it by hand:

```
: heat      HEATING set-mode ;
: cool      COOLING set-mode ;
: hvac-idle IDLE    set-mode ;
```

Notice that `heat`, `cool`, and `hvac-idle` don't touch `mode` at all — each is just a name for "``call set-mode with a particular constant.``" That wasn't planned in advance; it fell out once `set-mode` existed as its own word, because writing "what changes the mode" three different ways (once automatically, three times manually) would have meant deciding, three separate times, whether the change was worth acting on. One shared word underneath both the automatic and the manual path means that decision gets made once. This is the same move Brodie's book makes with an editor whose `INSERT` command turns out to be nothing more than "make room, then overwrite" — a word that already existed, doing a job nobody had thought to name yet. It's not something you plan for up front so much as something you notice once you've written the pieces down and looked at what they have in common.

A Change in Plan

Here's where a component-based design earns its keep. Suppose the requirement changes: instead of announcing the mode every cycle, the thermostat should only speak up when the mode actually *changes* — nobody wants a log line every ten seconds saying "still heating."

Because **set-mode** already sees both the old mode and the new one, it already *has* the information this change needs. Adding it costs one line, using one new word: **s:strfmt** takes a value and a format string containing a placeholder (**%s** for a string, **%d** for a number) and produces the finished string, value substituted in — “cooling” “now %s\n” **s:strfmt** leaves the string “now cooling\n” on the stack, ready to print:

```
[ "idle" , "heating" , "cooling" ] constant mode-names

: mode-name \ mode -- s
  mode-names swap caseof ;

: set-mode \ new-mode --
  dup mode@ n:= not if
    dup mode !
    mode-name "now %s\n" s:strfmt . \ <-- the new line
  else
    drop
  then ;
```

Nothing else in the program changes. **heat**, **cool**, **hvac-idle**, and **decide-mode** are untouched, and every one of them picks up the new, quieter logging behavior automatically, because every one of them was already going through **set-mode**.

Compare that to what would have happened if the mode had been changed directly wherever it was decided — three lines in the manual-override words, one more inside whatever called **decide-mode** — with a “print the mode” step tacked on after each one, the way a flowchart-driven design naturally accumulates: one box for “decide,” a separate box for “report,” wired together by the arrows between them. Adding “only report when it changed” to *that* design means the “did it change” check either gets duplicated at every call site, or the previous mode has to be threaded through the control flow as extra state so the reporting step can compare against it — extra plumbing whose only job is to reconnect two things that a shared word would have kept connected for free. The component version didn’t dodge that problem by being cleverer. It dodged it by already having exactly one place where the answer to “did the mode change?” was knowable without asking around.

[code/ch03/thermostat.8th](#) exercises this with a run of temperature readings. Reading the code rather than running it first: **60**, then **61**, then **72** should produce two mode changes and one silent repeat —

- **60** → HEATING (a change: **set-mode** logs **now heating**)
- **61** → HEATING again (no change: silent)
- **72** → IDLE (a change: **set-mode** logs **now idle**)

which is exactly what the actual run confirms further below, alongside a fourth reading this chapter isn't ready to explain yet.

Decomposition by Sequential Complexity, and Its One Wrinkle

The thermostat's words, read top to bottom, go from simplest to most capable: constants, then the mode variable, then **set-mode**, then the words built on top of it. That ordering isn't a style choice — 8th, like Forth, requires a word to be defined before anything can refer to it, so a program built up word by word naturally reads like a textbook, elementary material first.

Occasionally that ordering fights you. Suppose the sensor component needs to flag an implausible reading — a wildly out-of-range number that suggests a wiring fault — but the code that actually knows how to *handle* that situation (log it, alert someone, whatever a future diagnostics component decides to do) doesn't exist yet, and arguably shouldn't be designed until there's a real diagnostics story to design it around. The foundational sensor code is written first; the advanced response to it comes later. But the sensor code still needs to call *something* when it sees a bad reading, today, before that something has been written.

8th's answer to this is **defer:** — a word declared now, whose body is supplied later. The plausibility check below also uses **and**, which takes two booleans and is true only if both of them are:

```
defer: on-bad-reading    \ degrees --

: plausible? \ degrees -- flag
  dup -20 n:> swap 120 n:< and ;

0 var, sensor-temp
```

```
: read-temp \ -- degrees
  sensor-temp @
  dup plausible? not if dup on-bad-reading then ;
```

Until something is attached to it, **on-bad-reading** is silently a no-op — **read-temp** compiles and runs correctly with no diagnostics component in sight. Later, once that component is designed, it attaches itself with two new words. ' (a single quote, called “tick”) precedes a word’s name and pushes a *reference* to that word instead of running it — ' **report-bad-reading** puts a handle to **report-bad-reading** itself on the stack, not the result of calling it. **w:is** then takes that reference and a deferred word, and makes the deferred word run the given one from now on:

```
: report-bad-reading \ degrees --
  "sensor reading %d looks implausible -- check wiring\n"
s:strfmt . ;

' report-bad-reading w:is on-bad-reading
```

From this point on, every call to **read-temp** that sees an implausible value invokes **report-bad-reading** — without **read-temp** having been touched, and without the sensor component needing to know, when it was written, what “diagnosing a bad reading” would eventually mean. Running the same reading (999, well outside a plausible range) before and after this assignment shows the difference directly:

```
\ before the diagnostics component is wired in:
999 sensor-temp ! auto-cycle
\ => "now cooling"    (mode genuinely changes; no diagnostic – the
hook
\   is still a no-op)

\ after ' report-bad-reading w:is on-bad-reading :
999 sensor-temp ! auto-cycle
\ => "sensor reading 999 looks implausible -- check wiring"
\   (mode is already "cooling", so no "now ..." log this time --
\   only the diagnostic fires)
```

This is [code/ch03/thermostat.8th](#) in full; running it start to finish prints exactly:

```
now heating
now idle
now cooling
sensor reading 999 looks implausible -- check wiring
final mode: cooling
```

defer: is a narrow tool for a narrow problem — a genuine forward reference, not a general substitute for planning ahead — but it means the order you’d naturally *write* a program (foundations first) doesn’t have to match the order in which every dependency becomes known.

The Limits of “Level” Thinking

It’s tempting, once a program is split into “foundational” and “advanced” pieces, to treat that split as a hierarchy you must climb in order — design the bottom first, then the middle, then the top. Nothing about component decomposition requires that. The thermostat example above was written **mode/set-mode** first only because that made the clearest starting point for this chapter — but **decide-mode** could just as easily have been written and tested first, standing on nothing but plain numbers on the stack, long before **read-temp** or **sensor-temp** existed:

```
60 decide-mode . \ works today, no sensor required
```

Pick whichever piece gives you the most useful feedback fastest — the part you’re least sure about, the part a stakeholder most needs to see working, the part whose difficulty will tell you whether the rest of the project is easy or hard. “Foundational” and “advanced” describe where a word ends up in the finished dependency graph, not the order you’re obligated to design them in.

A related trap is building an interface too narrow to reach the thing behind it. If a component exposes only the three or four operations its first caller happened to need, and hides everything else — including information a *later* caller turns out to need but the component’s author never anticipated — the later caller is stuck. It can’t extend the component’s own tools, because it doesn’t have access to them; it can only work around the outside of a boundary that was drawn too tight. The fix isn’t to expose everything (that defeats the point of having a boundary at all) — it’s to expose the tools the component is built from, not just the one function that happened to be needed first, so a future caller with a slightly different need can compose those tools instead of reimplementing them from scratch outside the

boundary. **mode@** above is a small instance of this: it costs one line to expose, and it means a future component that only needs to *read* the mode — a display, a log, a scheduler — never has reason to reach past **set-mode** and touch the variable directly.

Brodie’s own version of this chapter, writing in 1984, extends the point into a broader argument against “objects” — meaning, in his usage, a single word that takes a selector parameter and internally dispatches to one of several behaviors, the way an old COM or CORBA object might. His concern was narrow and specific even though the term he used for it is broad: such a word has to contain its own internal decision structure to figure out which behavior you meant, and it can’t be extended by adding a new named word the way a namespace of many small words can — you have to modify the dispatcher itself. That’s a real, fair comparison between *that specific shape* (one name, an internal switch) and 8th’s usual shape (many names, no switch needed because the name already picked the behavior). It is not a fair comparison between 8th and object-oriented programming in general — most OOP languages built after 1984 don’t work by internal selector-dispatch either, and later editions of Brodie’s own book say as much. The lesson worth keeping is narrower than “objects are bad”: a word that has to ask “which of my several jobs am I doing this time?” is doing work that a well-factored set of separately named words doesn’t have to do — regardless of what you call the wider style it’s part of.

Summary

Preliminary design means deciding what pieces a program needs before writing them. 8th supports two ways of arriving at those pieces: grouping by what a piece knows (decomposition by component, which is where the real design work happens) and the ordering a word-based language imposes anyway (decomposition by sequential complexity, which **defer:** lets you escape when a foundational piece genuinely needs to call something that isn’t written yet). The thermostat example showed both at once: a shared **mode** component discovered by noticing that several separate callers wanted the same underlying action, an interface disciplined enough (**mode@**, **set-mode**) that a real requirement change cost one line instead of a redesign, and a forward-referenced hook that let a foundational word call forward into a component that didn’t exist yet. None of this required planning the whole dependency graph before writing any code — only writing each piece where its boundary was actually the right one to draw.

Chapter 4: Detailed Design and Problem Solving

Preliminary design tells you what components a program needs. Detailed design is where you actually solve each one — the part most programmers find the most fun, and the part where a chapter about *8th* has the least to add over a chapter about anything else, because solving problems isn't really language-specific. What *is* language-specific is what happens once you have a solution in mind: how naturally can you say it? This chapter covers both — a short, general toolkit for getting unstuck, and a longer look at how *8th*'s own shape steers you toward particular kinds of solutions — then puts both to work on one real problem, start to finish.

Getting Unstuck

None of the following is specific to programming, let alone to *8th*, but it's worth stating plainly because it's easy to forget under deadline pressure:

- **Know your goal before you start**, concretely enough to write it as a stack-effect comment. If you can't write the SED, you don't understand the problem yet.
- **Hold the whole problem in your head at once** before reaching for a partial solution. Fill your mind with the requirements the way you'd fill your lungs before diving, and see whether the shape of an answer appears on its own. Often it does.
- **When it doesn't, work backward**. Some problems are far easier to solve by assuming you've reached the end and asking what the last step must have been, than by pushing forward from the start. The classic example — measuring exactly six gallons using only a nine-gallon and a four-gallon container, with no markings on either — barely budes if you experiment forward from empty containers. Assume instead that six gallons is already sitting in the nine-gallon container, and ask what the *previous* state must have been. That question has only two possible answers, and one of them unravels the whole problem in a couple of steps.

- **Recognize the auxiliary problem.** Partway into a solution you’ll often notice a sub-problem that doesn’t quite belong to the main line of reasoning — a piece you clearly need but haven’t solved yet. Name it, assume it has a solution, and keep moving on the main problem. 8th makes this concrete rather than aspirational: Chapter 3’s **defer:** is exactly “assume a solution exists, wire it in by name, solve it later” turned into working code.
- **Step back when stuck.** Attachment to your first idea is the most common reason a solvable problem stays unsolved. If a problem feels impossible, check what constraints you’ve assumed that the problem never actually stated.
- **Don’t stop at the first working answer.** Ask whether a second pass would be simpler, not just whether the first one works.

Working backward isn’t unique to programming — it’s the same move that solves a maze faster from the exit than from the entrance:

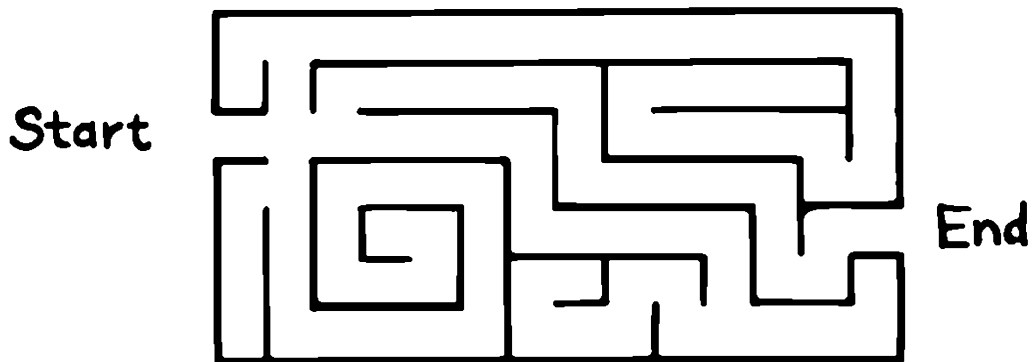


Figure 7: A hand-drawn maze labeled Start and End, with no other markings – a problem that’s genuinely easier to solve backward than forward.

and it’s exactly what solves the nine-and-four-gallon puzzle. Assume six gallons is already sitting in the nine-gallon container:

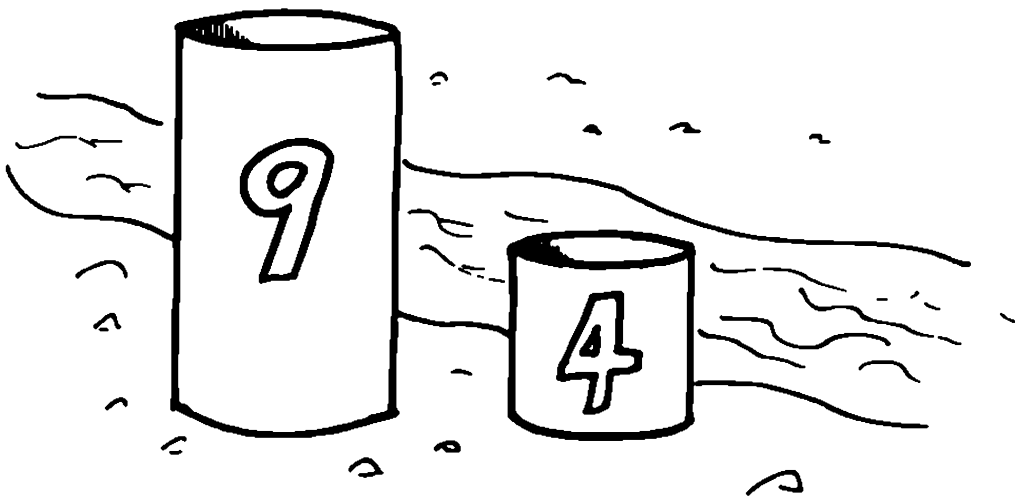


Figure 8: Two labeled containers, a nine-gallon one and a four-gallon one, sitting in a stream, both currently empty.

— then ask what the previous pour must have been. Only two answers are possible, and working through both at once makes it obvious which one actually unravels the problem:

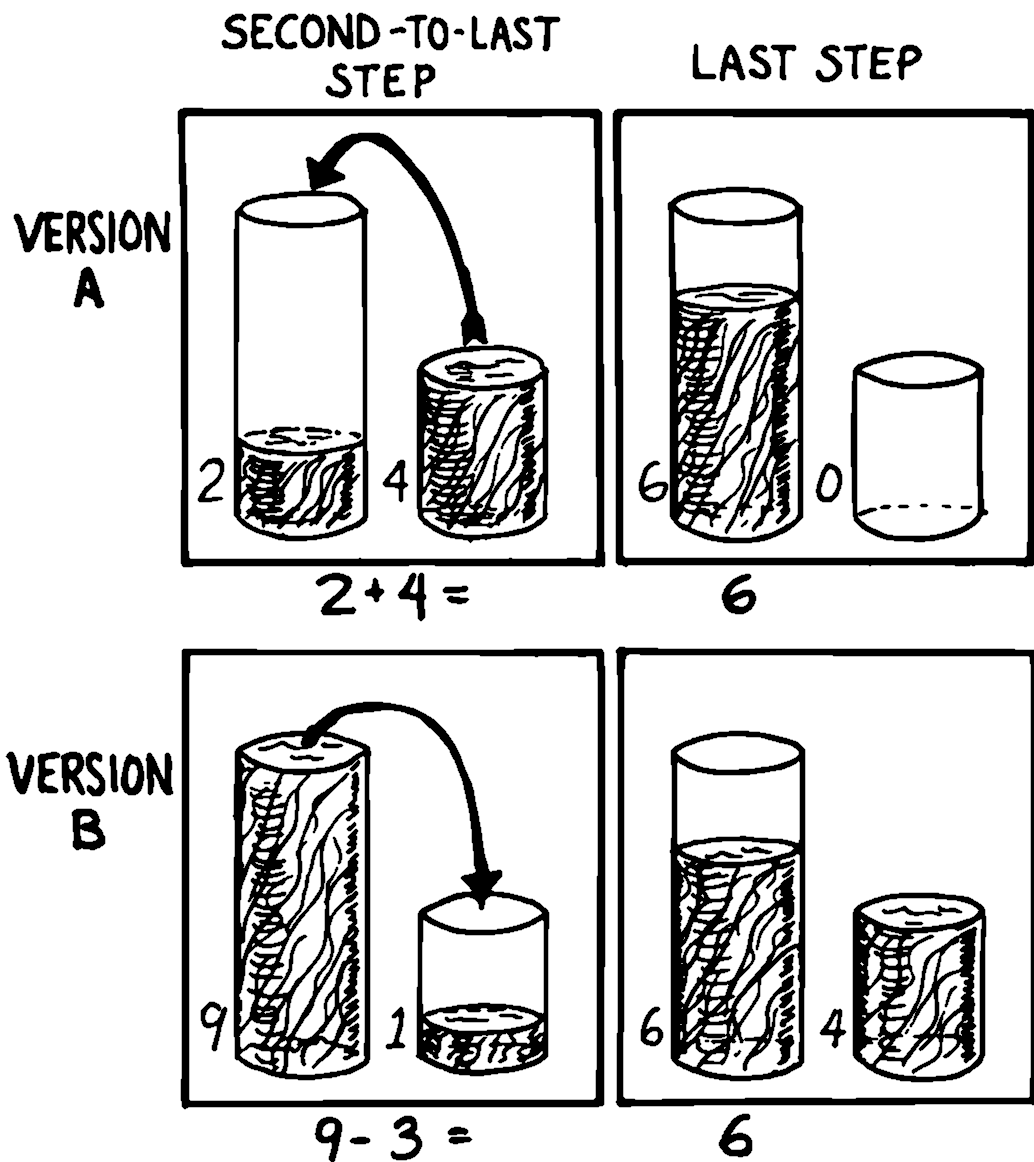


Figure 9: A two-by-two grid contrasting two backward-reasoning hypotheses: Version A pours 2 and 4 together to reach 6 and 0, which cannot be reached from empty containers by any legal pour; Version B pours 9 minus 3 to reach 6 and 4, which can.

Version B is the one that resolves: a nine-gallon container holding nine and a four-gallon container holding one is reachable in a couple of legal pours from two empty containers, so the whole solution unwinds backward from there.

Designing a Component

Once you know a component is needed (Chapter 3), designing it has a recognizable shape:

1. Decide the names and calling syntax of the words the rest of the program will see — the interface.
2. Work out the algorithm(s) and data structure(s) behind that interface.
3. Notice the auxiliary words this will require, and check what's already available before writing new ones.
4. Sketch the algorithm in pseudocode, then implement it, generally by working backward from the pieces you already have toward the raw input.

The rest of this chapter works through the first two of these in depth, then puts all four to use in one extended example.

How 8th Wants to Be Written

A stack-based, postfix language doesn't have much syntax to get wrong, but it has strong *conventions* — regularities that make code predictable to read even when you've never seen a particular word before. None of these are enforced by 8th; all of them are worth following anyway, because breaking them makes your words surprising to whoever reads them next, including you in six months.

Numbers, and anything else a word needs, come before the word. A word that expects a number pulls it from the stack, so the number has to already be there: **20 apples !**, not **apples ! 20**. This is just postfix notation, the same rule that makes **3 4 n:+** read strangely to newcomers and then stop feeling strange within about a day.

A name precedes text it introduces. "**a string**" is data sitting on the stack, or a name being defined, and either way it's the *word before it* that gives that text meaning — **constant**, **var**, **,**, **:**. There's no way for 8th to make sense of bare text on its own, so something naming it always comes first.

"Noun verb" beats "verb noun." **apples !** reads as "the apples slot, store" — a thing, then an action on it — and that ordering falls out naturally from the stack: the reference has to be pushed before the word that consumes it can run. Chapter 1's **apples**, **red**, **green** are all nouns and modifiers in exactly this sense, regardless of how they're implemented underneath.

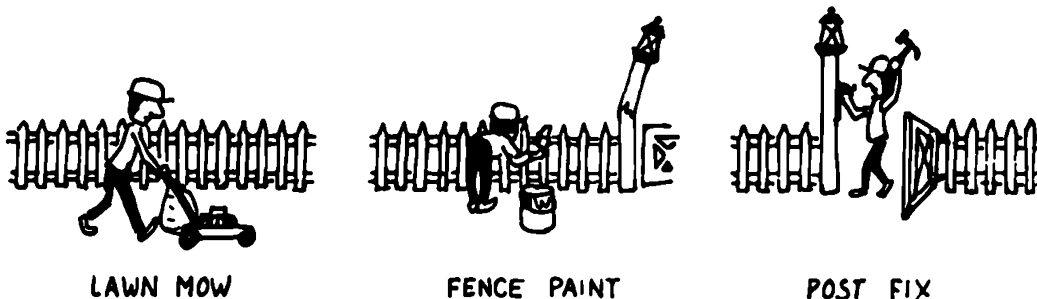


Figure 10: Three small vignettes of everyday yard work, each captioned noun-then-verb: a person mowing a lawn captioned *LAWN MOW*, a person painting a fence captioned *FENCE PAINT*, and a person fixing a post captioned *POST FIX*.

It’s not a computing convention so much as an everyday one made visible: say what you’re working on, then what you’re doing to it, the same order this book’s own examples have used all along.

Definitions consume the arguments they’re given, in full, even when that means an argument gets duplicated on the way in rather than smuggled back out. If three internal words each need a garage level number, put the `dup`s inside those three words, not in the word that calls all three — so each one reads correctly on its own, and the caller doesn’t need to know how many copies its argument requires:

```
: with-dup    dup n:1+ . cr ;
: without     n:1+ . cr ;

5 with-dup    \ prints 6, leaves 5 on the stack for whatever's next
```

Avoid words that look ahead at what comes next in the source. 8th gives you the tools to peek at the next token in the input stream and act on it — ‘ (tick) followed by **w:exec**, for instance — but a word that depends on *specifically what* follows it in the source is fragile: call it with the wrong thing after it, or from inside another definition where there’s no “next token” the way you expected, and it breaks in confusing ways. Chapter 3’s **defer:** / **w:is** solves the same underlying problem — “do something not yet known at definition time” — without that fragility, which is why it’s the 8th-idiomatic answer here rather than input-stream lookahead.

Container indices start at zero, and 8th’s own arrays already agree with you on this — there’s no “start counting from one” convention to fight. If a problem’s natural units start at one (item #1, not item #0), do the subtraction once, at the boundary where a person’s input becomes an internal index, rather than throughout your code.

Calculation, Data Structure, or Logic

Given a mapping from inputs to outputs, there are exactly three ways to build it, and they’re worth trying *in this order*:

1. **Calculation** — a formula.
2. **Data structure** — a table you look up.
3. **Logic** — a chain of conditions.

Calculation wins whenever a formula actually exists, because it’s the least code and the easiest to get right. Suppose parking garage levels need a minimum ceiling clearance, two feet taller per level up (level 1 needs 7ft, level 2 needs 9ft, and so on):

```
: clearance-ft \ level -- feet
2 n:* 5 n:+ ;

1 clearance-ft . cr \ => 7
2 clearance-ft . cr \ => 9
3 clearance-ft . cr \ => 11
```

One line, and it’s obviously correct for every level, not just the ones you tested.

A data structure wins when the mapping is real but *not* formulaic — a business decision, not a law of arithmetic. Chapter 2’s parking-fee rate table is exactly this case: nothing about weekday-evening pricing being \$2.00 rather than \$2.25 follows from a formula; it’s a rate someone set, looked up by tier through **caseof**. Trying to calculate it would mean inventing a formula that happens to fit today’s numbers and will happen to be wrong the next time the rates change.

Logic is last on purpose. It wins only when the decision genuinely depends on a combination of conditions that isn’t well described as either a formula or a lookup — for instance, whether a level is currently open to traffic, which might depend on the hour *and* whether there’s a maintenance flag set *and* whether a special event has reserved it. This is also a natural place for a new word, **;then:** shorthand for

“if this condition is true, exit the word right now” — an early return, useful for exactly this kind of guard-clause checklist:

```
false var, maintenance?  
false var, event-reserved?  
  
: level-open? \ hour -- flag  
  maintenance? @ if drop false ;then  
  event-reserved? @ if drop false ;then  
  8 n:< if false ;then  
  true ;
```

Read each guard the same way: “if this is true, leave **false** and return immediately; otherwise fall through to the next check.” Only if none of the three guards fires does execution reach the final **true**.

Logic isn’t wrong here — some things really are conditional — but reach for it last. A chain of `if`s is easy to write and hard to verify by inspection once it grows past three or four branches, in a way a formula or a table isn’t.

Solving a Problem: Roman Numerals

Time to put all of this to work on a real, complete component: a word that turns a number into a Roman numeral.

Interface first. The component needs exactly one externally-visible word. It takes a number and hands back a string:

```
: roman \ n -- s
```

Everything else is internal.

The algorithm. Look at how Roman numerals actually work: 1994 is MCMXCIV — a thousand (M), then nine hundred (CM), then ninety (XC), then four (IV). Each of those pieces is the largest Roman-numeral “chunk” that still fits in what’s left, written down, with its value subtracted before moving on to the next-largest chunk. That’s the whole algorithm: a descending list of (value, symbol) pairs, and a rule of “take as many of the largest chunk as fit, then move to the next one.” This is a calculation-vs-data-structure choice in exactly the sense of the previous section — there’s no formula for Roman numerals, but there’s an obvious table:

```
[ 1000 , 900 , 500 , 400 , 100 , 90 , 50 , 40 , 10 , 9 , 5 , 4 , 1 ]
constant roman-values
```

```
[ "M" , "CM" , "D" , "CD" , "C" , "XC" , "L" , "XL" , "X" , "IX" ,
  "V" , "IV" , "I" ]
constant roman-numerals
```

Thirteen chunks, largest first, each value paired by position with its symbol — position 0 is a thousand and "M", position 12 is one and "I". 900 and "CM" sit right after 1000/"M" rather than after 500/"D", which is what makes “nine hundred” come out as CM instead of DCCCC: the table already encodes the special-case shortcuts, so the algorithm on top of it doesn’t need to know they’re special cases at all.

Working backward from the pieces. The word that has to exist no matter what is “keep taking a given chunk while it still fits” — a repetition, which needs two new words. **repeat** marks the top of a loop; **while!** goes at the bottom, checking the boolean on top of the stack and jumping back to the matching **repeat** if it’s true, or falling through to whatever comes after if it’s false, consuming that boolean either way. On its own, **repeat ... while!** always runs its body at least once before the first check — but wrapping the whole thing in **if**, testing the same condition first, skips the body entirely when it isn’t needed even once:

```
0 var, remaining

: due? \ -- flag
  remaining @ 0 n:> ;

: count-down \ --
  due? if
    repeat
      remaining @ . cr
      remaining @ 1 n:- remaining !
    due?
  while!
  then ;

3 remaining ! count-down \ prints 3, 2, 1
0 remaining ! count-down \ prints nothing -- due? was already
false
```

`consume-tier` is the same shape, with two more tables to read from instead of one variable to count down:

```
0 var, remaining
"" var, result
0 var, tier

: value@    roman-values tier @ caseof ;
: numeral@  roman-numerals tier @ caseof ;

: append    \ s --
  result @ swap s:+ result ! ;

: due?      \ -- flag    (does this tier's value still fit in what's
left?)
  remaining @ value@ n:< not ;

: consume-tier \ --
  due? if
    repeat
      numeral@ append
      remaining @ value@ n:- remaining !
      due?
    while!
  then ;
```

`tier` names *which* row of the two tables is current — the same “current column” idea Chapter 1’s `apples` example used, here selecting a value/symbol pair instead of a red/green tally. `consume-tier` doesn’t care what tier it’s operating on; it just keeps appending that tier’s symbol and subtracting that tier’s value for as long as `due?` says yes, using the pre-checked loop shape (`if ... repeat ... while! ... then`) that tests *before* the first iteration, so a tier that doesn’t apply at all — `M` when only 4 is left — correctly does nothing.

The outer word just has to visit all thirteen tiers in order, letting `consume-tier` decide how many symbols each one contributes. `loop` is the last new word this example needs: give it a word reference (with `'`, the same tick you saw attach a diagnostics handler in Chapter 3), a low index, and a high index, and it calls that word once for every index in that range, inclusive, passing the index in each time:

```

: apply-tier \ index --
tier !
consume-tier ;

: roman \ n -- s
remaining !
"" result !
' apply-tier 0 12 loop
result @ ;

```

' **apply-tier 0 12 loop** calls **apply-tier** thirteen times, once per tier from 0 (a thousand) to 12 (one) — exactly the visiting order the algorithm needs, with no explicit counting.

Run against a spread of values, including the traditionally trickiest ones — **1994**, and **3999**, the largest number classical Roman numerals can represent at all:

```

1 roman . cr      \ => I
4 roman . cr      \ => IV
9 roman . cr      \ => IX
14 roman . cr     \ => XIV
40 roman . cr     \ => XL
49 roman . cr     \ => XLIX
90 roman . cr     \ => XC
444 roman . cr    \ => CDXLIV
1994 roman . cr   \ => MCMXCIV
3999 roman . cr   \ => MMMCMXCIX

```

```

I
IV
IX
XIV
XL
XLIX
XC
CDXLIV
MCMXCIV
MMMCMXCIX

```

Every one matches. This is [code/ch04/roman.8th](#), executed exactly as shown.

Note what didn't need to exist: no special-casing for “the 4 pattern,” no branch for thousands versus ones, no manual digit-by-digit decimal decomposition. All of that complexity moved into the *data* — thirteen rows instead of four decimal columns — which is a fair trade, because data is easier to double-check by reading it than logic is. If Roman numerals had a fourteenth irregular case, it would mean adding one row to two arrays, not touching `consume-tier` at all.

A caller who only ever needs numbers up to 3999 doesn't need to be told so by the code above — Roman numerals themselves don't represent anything larger by convention. Whether `roman` should guard against `4000` and refuse, or simply produce a very long string, is a decision about the *interface*, made consciously, not a bug to discover later.

Summary

Detailed design has a recognizable shape — decide the interface, then the algorithm and data structures behind it, solving auxiliary problems by naming and deferring them rather than solving everything at once. 8th carries its own strong conventions for how that design should read once written: numbers and arguments precede the words that consume them, definitions take full responsibility for the arguments they're given, and container indices start at zero because 8th's own containers already do. Given a mapping to implement, try calculation first, a data structure second, and logic last — not because logic is wrong, but because it's the hardest of the three to verify by eye once it grows. The Roman-numeral example put all of it to work at once: a data-structure-first algorithm, built backward from the smallest working piece, verified against exactly the cases most likely to expose a mistake.

Chapter 5: Elements of Style

Badly written Forth has been compared to code that went through a trash compactor, and 8th earns the same warning honestly: a language with this little enforced structure gives you the freedom to write something genuinely hard to read, right alongside the freedom to write something exceptionally clear. Nothing in the language nudges you toward either one. Style — how you organize source across files, how you comment, how you name things — is what decides which freedom you actually used.

This chapter is about that decision: organizing source, documenting stack effects precisely, and choosing names that pull their weight.

Organizing Your Source

In 1984, Forth stored source code in fixed-size “screens” — literally, disk blocks, addressed by number, loaded with directives like `-->` and `THRU` that told the system which screens to read next. A large chunk of Brodie’s original chapter is about disciplines for numbering and loading screens well. None of that exists in 8th, or in most other languages written since: 8th source lives in ordinary text files, of any length, organized however your filesystem lets you organize anything else. The mechanism is gone; the underlying question it was trying to answer isn’t: *given a program built from many small words across many components, how should the source be laid out so a reader can find their way around it?*

The answer Chapter 3 already gave for *words within one file* — let the order mirror the “uses” hierarchy, foundations before what’s built on them — extends naturally to *files within a project*: put each component in its own file, and load the files a component depends on before the files that use it.

8th’s word for this is `f:include`. Give it a file path (as a string, the same as any other), and it reads that file and runs it, exactly as if you’d typed its contents at this point yourself — which means every word the included file defines becomes available immediately afterward:

```
\ stock-component.8th
0 var, total

: stock:add    \ n --
  total @ n:+ total ! ;

: stock:count  \ -- n
  total @ ;
```

```
\ stock-main.8th
"code/ch05/stock-component.8th" f:include

5 stock:add
3 stock:add
stock:count . cr    \ => 8
```

This is `code/ch05/stock-main.8th` and `code/ch05/stock-component.8th`, run together exactly as shown, from the repository root.

That last detail matters and is easy to get wrong: `f:include`'s path is resolved relative to *the current working directory at the moment you run the program* — not relative to the file doing the including, the way some other languages' import statements work. `stock-main.8th` says "`code/ch05/stock-component.8th`", which only resolves correctly because this book's convention (established in "Getting 8th and Running Your First Program") is to always run examples from the repository root. Move either file, or run it from somewhere else, and the path needs updating to match — 8th won't figure out "the file next to me" on your behalf.

*(8th also has **needs**, which loads a file by name from a fixed set of library locations — worth knowing about, but a different tool: it's for pulling in a shared library that lives in one of those known places, not for organizing the files of your own application relative to each other.)*

Formatting: Spacing and Indentation

Nothing in 8th enforces indentation — you could write every example in this book as one unbroken line and it would run identically. Every example in this book has nonetheless been indented consistently: `if/else/then` bodies indented one level in, a `repeat/while!` body indented the same way, a word's body indented under its `:` line. None of that was called out explicitly until now, because the pattern itself was the lesson — by Chapter 3 or so, the indentation was probably already telling you where a conditional's branches began and ended before you'd consciously noticed it doing so. That's what consistent formatting is *for*: structure a reader can absorb without deliberately parsing it.

Documenting Stack Effects, Precisely

"A Note on Notation" introduced the shape of a stack-effect comment — `\ inputs` — `-- outputs` — but not a shared vocabulary for describing what those inputs and outputs *are*. A small set of one- or two-letter abbreviations, used consistently, makes a SED readable at a glance without reading the word's body at all:

abbreviation	means
n	a number
s	a string
flag	a boolean
a	an array
m	a map
ref	a variable reference (what a var ’s bare name produces)

Every SED already shown in this book happens to follow this vocabulary — `\ hours -- cents` and `\ level -- feet` are really `n -- n` with more specific names substituted where the specific meaning mattered more than the type. Both are legitimate; use the specific name when it adds real information, the generic abbreviation when the type is all that matters.

Some words don’t have *one* stack effect — they have one for each outcome. Chapter 4’s `level-open?` always returns exactly one flag, but plenty of realistic words return different *numbers* of things depending on what happened. A division that has to handle “don’t divide by zero” is a small, honest example:

```
: safe-div \ n1 n2 -- n1/n2 true | false
  dup 0 n:= if
    drop drop false
  else
    n:/ true
  then ;

10 2 safe-div . cr . cr    \ => true, then 5
10 0 safe-div . cr        \ => false
```

```
true
5
false
```

Read the SED’s `|` as “or”: either the division succeeds and you get the quotient followed by **true**, or it doesn’t and you get **false** alone — two genuinely different stack pictures for the same word, both written out in full, because writing only one of them would silently promise something that isn’t always true.

Choosing Names

Short names are easier to read than long ones — *if* they’re still clear. **space-available?** earns its length because “free?” or “ok?” would have been too vague to be worth the four fewer characters; **.** earns its brevity because “print” would add nothing that context doesn’t already supply. There’s no formula for this, only the test of reading the finished line back and asking whether a stranger would understand it.

One instinct worth resisting: naming a word to describe everything it does, rather than what it *is*. A prefix or suffix is for *distinguishing* one word from a similarly-named other, not for cramming a description into the name itself. **garage-full?** doesn’t need to be **garage-currently-at-maximum-capacity-flag** — the **?** alone tells a reader “this is a question, expect a boolean back,” which is exactly as much as the name needs to promise.

That **?** is doing real, and by now familiar, work. Look back over this book’s own examples: **hurried?**, **space-available?**, **due?**, **plausible?**, **level-open?** — every predicate this book has written, without the convention ever being named until now, ends in **?**. That’s not a coincidence and it isn’t unique to this book; it’s close to universal practice across Forth and 8th alike, precisely because it costs one character and answers a real question — “can I use this directly in an **if**?” — before you’ve read a single line of the definition.

8th gives this instinct a second, formal outlet beyond suffixes: namespaces. Chapter 1 already made the case that 8th’s built-in **n:**, **s:**, **a:** are namespaces doing, as an enforced language feature, what Forth programmers used to do only by naming discipline. The same tool is available for your own components — nothing restricts namespace prefixes to 8th’s own built-in words:

```
: stock:add    \ n --
  total @ n:+ total ! ;

: stock:count  \ -- n
  total @ ;
```

stock:add and **stock:count**, from the file-organization example above, already used this: the **stock:** prefix says, at the call site and not just in the defining file, which component a word belongs to — the same information a suffix like **?** gives about a word’s *return type*, now given about its *component membership* instead.

Between namespace prefixes for “which component” and suffixes like ? for “what kind of answer,” 8th’s own naming toolkit covers most of what Forth programmers used to negotiate by convention alone.

Summary

8th’s freedom from enforced structure doesn’t make style optional — it makes style the entire difference between readable and unreadable code, since nothing else will supply that difference for you. Organize source files the way Chapter 3 already taught you to organize words within one file: foundations first, loaded with `f:include`, paths resolved relative to where the program runs from rather than where the including file lives. Format consistently enough that structure is visible before it’s read. Write stack-effect comments with a shared vocabulary of abbreviations, and write out every genuinely different outcome a word can have rather than picking one to document and hoping the rest are close enough. And name things to distinguish, not to describe — a ? suffix or a namespace prefix carries real information in one or two characters; a long descriptive name usually carries less than it looks like it does.

Chapter 6: Factoring

Chapter 3 was about decomposition *before* you write anything — cutting a problem into components while it’s still a plan. This chapter is about the smaller, more frequent decisions that happen *while* you write, and afterward: noticing that a piece of what you just typed deserves its own name, and giving it one. Brodie called this factoring, borrowing the word from arithmetic in exactly the sense you’d expect — pulling a common piece out of an expression so it only has to be written, read, and changed once.

A fair amount of Brodie’s original chapter is about disciplines that no longer apply for reasons Chapter 5 already covered: numbered screens are gone, and with them a whole vocabulary of habits for deciding what belongs on which screen. What’s left, once that’s set aside, is mostly language-independent judgment — the same judgment any programmer needs, in any language, about when a piece of code has earned its own name. A few of Brodie’s specific mechanisms don’t survive the move to 8th unchanged, and this chapter says so plainly where that happens, rather than pretending a translation exists where it doesn’t.

Factoring Out a Calculation

The simplest kind of factoring is also the easiest to miss: an expression that shows up more than once, or that you find yourself describing in a comment because the code alone doesn't say what it means. Either signal is worth acting on the same way — give the expression a name.

Suppose a sensor channel is considered to be approaching its limit once a reading passes two-thirds of the channel's rated maximum. Written inline, “two-thirds of” is just an expression:

```
900 2 n:* 3 n:/ . cr    \ => 600
```

— which is fine once, but if it appears at every place a warning level gets computed, both the meaning (“this is a warning threshold”) and the arithmetic itself (“multiply by two, divide by three”) are being repeated. Factor it into a word, and both problems disappear at once:

```
: two-thirds \ n1 -- n2
  2 n:* 3 n:/ ;

900 two-thirds . cr    \ => 600
```

This is [code/ch06/warning-level.8th](#), run and verified against the output shown.

One thing worth knowing before you lean on `n:/` for a ratio like this: it's true division, not the truncating integer division some other languages default to. `1000 two-thirds` — that is, `1000 2 n:* 3 n:/` — comes out to `666.66667`, a float, since 2000 doesn't divide evenly by three. That's not a bug in `two-thirds`; it's `n:/` doing exactly what it's documented to do. If a whole-number result matters, either the inputs need to divide evenly, or an explicit rounding step belongs after the division — `two-thirds` itself shouldn't quietly assume one or the other.

Don't Push a Decision Down as a Flag

A subtler version of the same idea applies to control flow, not just arithmetic. Suppose closing up for the day means locking the register and turning off the lights, but closing up on the last day of the week also means filing a report in between.

The tempting first draft passes a flag into one word and lets it decide internally what to do:

```
: bad-checklist \ end-of-week? --
  "lock the register\n" .
  if
    "file the weekly report\n" .
  then
    "turn off the lights\n" . ;

true bad-checklist \ => lock, file report, turn off
false bad-checklist \ => lock, turn off (no report)
```

This works, but every caller now has to know that a boolean controls an internal branch inside somebody else's word — the decision about *whether* to file a report has been separated from the code that actually knows *when* the week ends. Factor out the part that's genuinely shared, and let each caller supply its own extra step instead of threading a flag through the shared word to ask for it:

```
: closing-checklist \ --
  "lock the register\n" .
  "turn off the lights\n" . ;

: end-of-day
  closing-checklist ;

: end-of-week
  closing-checklist
  "file the weekly report\n" . ;

end-of-day
end-of-week
```

```
lock the register
turn off the lights
lock the register
turn off the lights
file the weekly report
```

This is [code/ch06/closing-checklist.8th](#), both versions, run and verified together. Nothing here calls `closing-checklist` with an argument telling it what kind of day it is — `end-of-day` and `end-of-week` each know that themselves, which is exactly where that knowledge belongs.

Factoring a Repeated Decision Into Data

“A Method for Design” already showed `caseof` looking a plain value up by position — a fee, a rate, a name. The same mechanism factors out a *decision*, not just a value, once you notice that `caseof` calls whatever it finds if that happens to be a word rather than data. A chain of “if the channel number is this, do that; if it’s this other one, do this other thing” is exactly the shape a growing `if/else` ladder takes on as more channels are added — and exactly the shape an array of word references replaces with one lookup:

```
: check-pressure      "check pressure\n" . ;
: check-temperature   "check temperature\n" . ;
: check-flow          "check flow\n" . ;

[ ' check-pressure , ' check-pressure , ' check-temperature , '
  check-temperature ,
  ' check-flow , ' check-flow , ' check-flow , ' check-flow ]
constant channel-actions

: alarm! \ channel# --
  channel-actions swap caseof ;

0 alarm! \ => check pressure
2 alarm! \ => check temperature
7 alarm! \ => check flow
```

```
check pressure
check temperature
check flow
```

This is [code/ch06/channel-alarms.8th](#), run and verified. The `'` (tick) here is the same one Chapter 4’s `roman` used to hand `loop` a word to call repeatedly — a reference to a word, rather than a call to it, so it can be stored in the array and invoked later. Adding a ninth channel with its own action means adding one entry

to **channel-actions**, not one more branch to a chain that was already getting hard to read.

Factoring Out Names: A Real Array, Not Parallel Variables

Brodie’s version of this problem was eight scores tracked in eight separately-named variables — **0STS**, **1STS**, and so on — which works until you need to look one up *by number*, at which point nothing in the names themselves helps: you’re back to an **if/else** chain just to turn a channel number into the right variable. His fix was a Forth mechanism called **CREATE...DOES>**, which builds a new *kind* of defining word — here, one that lays out a block of memory and hands back a word that indexes into it by number, so **ARRAY THRESHOLDS** would create a whole family of numbered cells behind one name.

8th has no **CREATE/DOES>**, and no general equivalent for building a new defining word — this isn’t a gap in the language so much as a different starting point. Forth needed that mechanism because a **VARIABLE** is one raw memory cell with nothing indexed about it at all; building an indexable table out of raw cells was worth a whole technique. 8th’s arrays already *are* that indexable table, as a native value you can hold, pass, and index directly — so the problem **CREATE...DOES>** was solving doesn’t come up in the same form. What’s new here is only the second half of the picture: Chapter 2 only ever built arrays with **constant**, to hold fixed lookup tables. A set of thresholds that can be *changed* — set once at start-up, then adjusted later — needs a **var**-held array instead, and a way to write into it by index:

```
[ 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 ] var, thresholds

: threshold@   \ channel# -- n
  thresholds @ swap caseof ;

: set-threshold! \ a channel# n --
  a:! drop ;

thresholds @ 0 100 set-threshold!
thresholds @ 3 250 set-threshold!

0 threshold@ . cr    \ => 100
```

```
3 threshold@ . cr    \ => 250
1 threshold@ . cr    \ => 0
```

```
100
250
0
```

This is [code/ch06/thresholds.8th](#), run and verified. `threshold@` is nothing new — the same `caseof` read Chapter 2 introduced, on an array that happens to be mutable instead of a `constant`. `set-threshold!` introduces `a:!`, whose stack effect is `array index value -- array`: it mutates the array in place and hands the same array back, which is why `set-threshold!` ends with `drop` — the caller here already holds the array in `thresholds`, so the returned reference isn’t needed. Getting the index and the value the wrong way round (`a n x`, not `a x n`) is a real trap: `a:!` reports it as **Expected Array but got Number**, which points at the symptom rather than the actual mistake, so it’s worth writing this word carefully once and trusting it afterward. One array, indexed by channel number, replaces what would otherwise be eight identical-looking variables and whatever chain of logic picked among them.

When to Factor

Brodie’s chapter spends a long section on criteria for *when* a piece of code has earned its own name — heuristics rather than rules, since no mechanical test settles it, and several of the sharpest ones come straight from how Charles Moore describes his own practice. Asked how long a Forth definition should be, Moore’s answer was blunt: about a line — “short words give you a good feeling.” That rule of thumb, more than anything Brodie adds to it, is what anchors the first heuristic below. The ones that hold up regardless of language are worth keeping, in roughly this form:

- **Factor at the point you feel unsure**, or where the logic starts to push past what you can hold in your head at once. That feeling is itself the signal — waiting for a more “objective” reason to appear usually means factoring later than you should have.
- **Factor where you’d otherwise want to write a comment**. A comment explaining what a block of code does is often a sign that the block wants a name of its own instead — the name replaces the comment and stays attached to the code through every future edit.

- **A factored word should do one thing**, describable without “and” — `check-pressure` reads its own sensor and reports; it doesn’t also reset an alarm.
- **Look for repetition of pattern, not just repetition of exact code.** `bad-checklist`’s two call sites didn’t repeat any text, but they repeated a *shape* — “run the shared steps, then maybe one more” — which was the actual thing worth factoring out.
- **Be sure you can name what you factor.** If nothing shorter and clearer than a restatement of the code comes to mind, that’s often a sign the boundary is in the wrong place, not that naming is hard.
- **Factor to hide the parts most likely to change.** `channel-actions` isolates *what each channel does* from *how a channel gets dispatched to*, so a new alarm behavior never touches `alarm!` itself.
- **Don’t factor for the sake of factoring.** A single three-line word used once, with no repeated shape and no unclear logic, doesn’t need to become two words — that just adds a name to remember for no corresponding gain in clarity.
- **Make today’s version work; improve it tomorrow.** Factoring is usually easier once the working shape of the problem exists in code, not before — resist the urge to guess at the “right” boundaries before you’ve written something to draw them against.

Factoring at Compile Time

One more kind of factoring doesn’t touch runtime behavior at all: 8th reads and runs source top to bottom, so anything written as ordinary words — arithmetic included — runs as the file loads, not later. That means a value derived from other values can be *computed* once, by 8th itself, instead of pre-calculated by hand and copied in as a bare number:

```
8 constant wide
4 constant ave

wide 3 n:* ave 2 n:* n:+ 80 swap n:- 2 n:/ constant leftmargin

leftmargin . cr          \ => 24
```

`leftmargin` centers a row of boxes on an 80-column display: three widths plus two gaps subtracted from 80, split in half. Writing `24` directly would work today, but it

would silently go stale the moment **wide** or **ave** changed — nothing would connect the two. Deriving it from **wide** and **ave** means it can never drift out of sync with the values it depends on, at no runtime cost, since none of this arithmetic happens again once the file has loaded.

The same idea applies to a table's own size:

```
[ 10 , 20 , 30 , 40 ] constant points
points a:len constant #points

#points . cr          \ => 4
```

#points can never disagree with **points**, because it's derived from **points** rather than counted by hand and written down separately — the same role Chapter 2's **caseof** tables play, one step further: **#points** there was **points** describing its own size.

This is [code/ch06/boxes.8th](#), run and verified against both outputs shown above.

Summary

Factoring is the habit of continually asking, while writing and while revising, whether a piece of code has earned its own name — a repeated calculation, a repeated shape of control flow, a repeated decision, a family of near-identical names that really wanted to be one indexed structure. 8th inherits most of Forth's judgment about *when* to factor unchanged; where it differs is in *what's available* to factor into. **caseof** reused from Chapter 2, now dispatching to word references instead of returning plain data, replaces a growing **if/else** chain. A native array, mutable through **var**, and written with **a:!**, replaces what Forth needed a whole **CREATE...DOES>** mechanism to build. And because 8th evaluates ordinary arithmetic as the file loads, deriving one constant from another, or a table's length from the table itself, costs nothing at runtime while keeping values that depend on each other from silently drifting apart. None of this is a reason to factor everything on sight — the same chapter that gives the criteria for factoring also gives the criterion for stopping: make it work today, and don't factor for the sake of factoring.

Chapter 7: Taming the Stack

Every example so far has passed arguments to words the same way: on the stack, implicitly, the way a pronoun stands in for a noun already mentioned. It works because most words need only one or two things at once. This chapter is about what happens when that stops being true — when a calculation genuinely needs more values in play than the stack comfortably carries — and about the handful of tools 8th gives you for that situation, several of which don't exist in the form Forth programmers had to make do with.

How Deep Is Too Deep?

Two or three items on the stack is about as much as a reader can track in a single definition without losing the thread — which of the three is which, and in what order they arrived. Brodie's original chapter made this point using **ROT**, **PICK**, and **ROLL**, Forth's tools for reaching below the top couple of stack items, and warned against leaning on them: needing to reach that deep is usually a sign that the values in question aren't really "the stack's business" anymore. 8th's own documentation makes almost the identical point, independently, about its own **pick** and **roll**: *"If you have so many elements on the stack that you need **pick**, those elements should be in an array instead."* That's not a coincidence — it's the same judgment, arrived at twice, about what a stack is actually for.

8th makes that array conversion a one-word operation. **a:close** takes a count and that many items already on the stack, and bundles them into an array in one step:

```
10 20 30 40 4 a:close . cr    \ => [10,20,30,40]
```

This is [code/ch07/collapse-array.8th](#), run and verified. Once several values are travelling together as one array instead of as separate stack items, they're subject to everything Chapter 6 already covered — indexed by **caseof**, mutated with **a:!** — without any custom bookkeeping to get there.

The same native-array habit answers a question Brodie spent a whole section on: how do you build yourself an auxiliary stack, separate from the one built into the language, for values that need genuinely stack-like (last-in-first-out) handling of their own? In Forth this took a hand-rolled **CREATE/ALLOT** block with manual

pointer arithmetic. In 8th, an array already behaves this way — **a:push** adds to the top, **a:pop** removes from it:

```
[ ] var, mystack

mystack @ 1 a:push mystack !
mystack @ 2 a:push mystack !
mystack @ 3 a:push mystack !

mystack @ a:pop swap mystack ! . cr    \ => 3
mystack @ a:pop swap mystack ! . cr    \ => 2
```

This is [code/ch07/mini-stack.8th](#), run and verified. The **swap** before each **mystack !** is worth noticing: **a:pop** leaves the shrunken array *underneath* the value it removed, so storing the array back requires bringing it to the top first — a small, real example of exactly the kind of stack bookkeeping this chapter is about escaping wherever it isn’t strictly necessary.

Escaping the Stack: Word-Local Variables

Some values genuinely don’t fit the “mention once, refer to as it” pattern — they’re needed more than once, by name, spread across a whole calculation. Brodie’s example was a box-drawing word taking four coordinates, each needed twice (once per corner it touches). His fix was ordinary Forth variables, used carefully within one definition — which worked, but only by convention: a Forth **VARIABLE** is always global, so nothing stopped some other word from reading or clobbering it, whether or not that was ever intended.

8th has a mechanism Forth didn’t: variables genuinely scoped to one word. Prefix the definition with **locals:**, and inside it, **w:!** and **w:@** set and fetch a named slot that exists only for the duration of that call:

```
locals:
: midpoint \ x1 y1 x2 y2 -- xm ym
  "y2" w:! "x2" w:! "y1" w:! "x1" w:!
  "x1" w:@ "x2" w:@ n:+ 2 n:/
  "y1" w:@ "y2" w:@ n:+ 2 n:/ ;

0 0 10 20 midpoint . cr . cr    \ => 10, then 5
```

This is [code/ch07/midpoint.8th](#), run and verified. `midpoint` unloads all four arguments into named slots up front — `w:!` takes the value and then the name, so the four lines above read bottom-to-top as “y2, then x2, then y1, then x1,” unloading the stack in reverse of how the arguments arrived — and the two `n: +/n:/` lines that follow read exactly like the arithmetic they are, each coordinate referred to by name instead of by stack position.

The word `locals:` is doing real work in that snippet — it isn’t decoration. Leave it off and `w:@/w:!` have no scope to work in. Unlike a `var,,` which creates exactly one storage location no matter how many words end up calling it, each word that opts in with `locals:` gets its own private set of named slots, invisible to any other word — including one of the same name declared elsewhere. There is no dictionary entry to pollute, no risk of the “Redefining” warning Chapter 1’s `apples` example triggered, because nothing named `x1` or `y2` exists once `midpoint` returns.

An Auxiliary Stack for One Temporary Value

Occasionally a value needs to be set aside for a moment — saved, overridden, then put back — without cluttering a variable that has to be declared somewhere far from where it’s used. Forth’s answer was the *return* stack: the same stack the language uses to remember where a word should resume after a call returns, borrowed for a second, unrelated purpose via `>R` and `R>`. Brodie’s chapter spends several pages on the discipline this demands, because the borrowing is real — push one temporary value without popping it before the word ends, and you’ve corrupted the address the system needs to get back to its caller. Every `>R` needs a matching `R>` on every possible path through the word, including paths that don’t look obvious at a glance.

8th sidesteps the entire hazard. `>r`, `r>`, and `r@` do give you a place to stash a value — but it’s a separate stack from the one managing actual calls and returns, kept apart precisely so a mismatched push or pop can’t corrupt control flow. The old discipline (balance your pushes and pops) still matters for keeping *your own* values straight, but the worst-case consequence Brodie warns about — silently scrambling the entire program’s flow of control hours after the bug was introduced — isn’t possible here by design.

```
true var, verbose?
```

```
: log \ s --
```

```

verbose? @ if . cr else drop then ;

: quietly \ --
  verbose? @ >r
  false verbose? !
  "should not print" log
  r> verbose? ! ;

true verbose? !
"before" log      \ => before
quietly
"after" log       \ => after
verbose? @ . cr  \ => true

```

```

before
after
true

```

This is [code/ch07/quietly.8th](#), run and verified. `quietly` saves whatever `verbose?` was — not necessarily `true` — forces it off for one call, then restores exactly what it found. Whatever the setting was before, it’s the setting again afterward.

When a Save/Restore Turns Out to Be Bad Factoring

Not every “save the old value, then restore it later” urge is actually solving the right problem. Sometimes it’s a symptom of a word that shouldn’t have been touching shared state to begin with.

Suppose a global tracks how many holes the *current* game is — nine for a short round, eighteen for a long one — and a word called **game** reads that global and plays that many holes, however “playing a hole” is actually implemented.

Later, a second need shows up: play some *specific* number of holes, without disturbing whatever **#holes** is currently set to for the game in progress. The tempting fix saves the old value, sets the new one, runs the game, and restores what was there — more `>r`-style bookkeeping solving a problem that arguably shouldn’t exist. The word that actually needs fixing is the one that hard-codes a dependency on the global in the first place:

```

: play-hole \ index --
  "playing hole " . . cr ;

locals:
: holes \ n --
  "n" w: !
  ' play-hole 0 "n" w:@ n:1- loop ;

0 var, #holes
: long 18 #holes ! ;
: short 9 #holes ! ;
: game #holes @ holes ;

short
game

```

```

playing hole 0
playing hole 1
playing hole 2
playing hole 3
playing hole 4
playing hole 5
playing hole 6
playing hole 7
playing hole 8

```

This is [code/ch07/holes.8th](#), run and verified. **holes** takes its count as an ordinary argument and never reads **#holes** at all — it’s **game**, and only **game**, that reads the current setting and hands it along. Playing an arbitrary number of holes needs no save, no restore, and no new machinery: **5 holes** just works, because **holes** was never entangled with “the current game” to begin with. When a save/restore feels necessary, it’s worth asking first whether the word doing the saving even needed to touch the shared value directly.

Sharing One Component for Two Purposes

Sometimes reusing the same piece of machinery for two different jobs is the right call — a single “am I in quiet mode?” check, say, guarding output that two unrelated parts of a program both need to suppress at different times. Brodie’s rule for when this is safe is a genuinely portable one: reuse a component for a second purpose

only if every use is mutually exclusive, and each one restores exactly what it found when it's done — which is precisely what the earlier **quietly** example did for a single, non-overlapping case.

The harder case is when the *uses themselves can nest*. A plain on/off flag breaks the moment two callers overlap: the inner one finishes, flips the flag back to “visible,” and silently un-suppresses output that the outer caller still needed suppressed. A counter, not a flag, is what makes nesting safe — each caller adds one on the way in and removes one on the way out, and only reaching zero means every caller has actually finished:

```
0 var, quiet-depth

: log \ s --
  quiet-depth @ 0 n:= if . cr else drop then ;

: quiet!    1 quiet-depth n:++! ;
: unquiet!  -1 quiet-depth n:++! ;

"a" log
quiet!
  "b" log
  quiet!
    "c" log
    unquiet!
    "d" log
  unquiet!
  "e" log
```

```
a
e
```

This is [code/ch07/quiet-depth.8th](#), run and verified. “b”, “c”, and “d” are all suppressed — the inner **quiet!** raises the depth to 2, and the matching **unquiet!** only brings it back to 1, so **log** stays silent through “d” as well, correctly reflecting that the outer caller was never done. Only the final **unquiet!**, bringing the depth back to 0, lets **log** speak again.

Summary

The stack is the right tool for arguments a word uses once and passes along, the way a pronoun stands in for something just mentioned — but it stops being the right tool the moment values need to be referred to by name, held onto across a calculation, or shared safely between unrelated pieces of code. 8th gives you more ways out of a crowded stack than Forth did: **a:close** and native array push/pop for grouping values that travel together, word-local variables (**locals:**, **w:@**, **w:!**) for values that need real names scoped to exactly one word, and an auxiliary **>r/** **r>** stack that borrows the shape of Forth’s classic trick without its classic danger, since it isn’t the same stack the language uses to find its way home. None of this replaces judgment: a save-and-restore is sometimes the right move, and sometimes a sign that a word is reaching for shared state it never needed to touch, and a component shared for two purposes is safe exactly as far as its uses stay mutually exclusive, or — if they can nest — as far as a counter, not a flag, is tracking how many of them are still in flight.

Chapter 8: Bundling State, Redirecting Behavior

Chapter 7 covered the stack side of Brodie’s “Handling Data: Stacks and States” — when to get off the stack, and how. This chapter covers the other half: what to do once several related values need to travel together as one thing, and what to do when a word’s *behavior*, not just its data, needs to change at runtime.

A Table of Related Values

Some state doesn’t come as a single number or flag — it comes as a handful of related values that only make sense together. A box being drawn on screen needs a top, a bottom, a left, and a right edge; change one without the others and the box is nonsense. Brodie’s Forth had no built-in container that could hold a bundle like that under one name, so his solution was a small defining word (**POSITION**) that carved individual named cells out of a shared block of raw memory — a substantial piece of machinery, built solely to give six numbers one collective identity.

8th’s **map** already is that collective identity. A **{ key: value, ... }** literal builds one, and **m:@** / **m:!** read and write it by name:

```
{ "top" : 0 , "bottom" : 0 , "left" : 0 , "right" : 0 } var, box

box @ "top" 10 m:! drop
box @ "top" m:@ . cr drop    \ => 10
```

10

This is [code/ch08/box-map.8th](#), run and verified. Two things about `m:!` and `m:@` are worth calling out before they surprise you: `m:!`'s stack effect is **map key value -- map** — it hands the map back rather than leaving nothing, and `m:@`'s is **map key -- map value** — it leaves the map sitting underneath the value it found. Both are the same convention Chapter 6 already showed for `a:!`: a container word gives you the container back so a chain of operations can keep going, which means a value fetched or stored in isolation, as above, leaves that container behind on the stack needing an explicit `drop` once you're done with it.

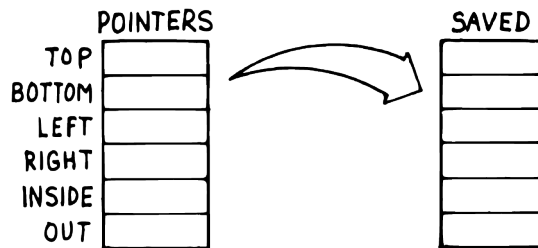


Figure 11: Two labeled columns of boxes, *POINTERS* and *SAVED*, each holding the same six field names – *TOP*, *BOTTOM*, *LEFT*, *RIGHT*, *INSIDE*, *OUT* – with a curved arrow copying the values from one column into the other.

Bundling four numbers into one map is only half of Brodie's actual problem, though. His real motivation was *saving and restoring* a bundle like this — trying out a change, then either keeping it or throwing it away. Suppose resizing the box is a two-step affair: begin a resize, adjust things freely, and only commit the result if it's wanted.

```
box @ G:clone var, draft drop

: begin-resize    \ --
  box @ G:clone draft ! ;
```

```
: commit-resize \ --
draft @ G:clone box ! ;
```

G:clone is the key new word here: given any value, it hands back a genuinely independent copy — for a container, everything inside is copied too, not just the outer reference. That distinction matters immediately. If **begin-resize** had written **box @ draft !** instead, **draft** and **box** would end up pointing at the *same* map, and editing one would silently edit the other — exactly the bug Brodie’s own **CMOVE**-based table copy was built to avoid, translated to 8th’s own easy way to get it wrong.

```
box @ "top" 10 m:! drop
box @ "bottom" 50 m:! drop

begin-resize
draft @ "top" 999 m:! drop

box @ "top" m:@ . cr drop      \ => 10
draft @ "top" m:@ . cr drop    \ => 999

commit-resize
box @ "top" m:@ . cr drop      \ => 999
```

```
10
999
999
```

This is [code/ch08/draft-commit.8th](#), run and verified. While the draft is being edited, **box** never changes — the two maps are independent copies from the moment **G:clone** made them. Only **commit-resize**’s own clone-and-store folds the draft’s values back in. Discarding a draft instead of committing it needs no special “cancel” word at all: just stop calling **commit-resize**, and **box** was never touched.

Two Live States, One Set of Names

A subtler version of the same problem shows up when there isn’t a “real” and a “draft” — there are two states that are both permanently live, and code needs to read and write “the current one” without caring which one that is at the moment. Brodie’s version alternated between a **REAL** table and a **PSEUDO** one; a thermostat

with separate comfort and energy-saving setpoints, switchable at will, is the same shape:

```
{ "heat-to" : 68 , "cool-to" : 76 } var, comfort
{ "heat-to" : 62 , "cool-to" : 80 } var, eco

comfort @ var, active-profile

: use-comfort    comfort @ active-profile ! ;
: use-eco        eco @ active-profile ! ;
```

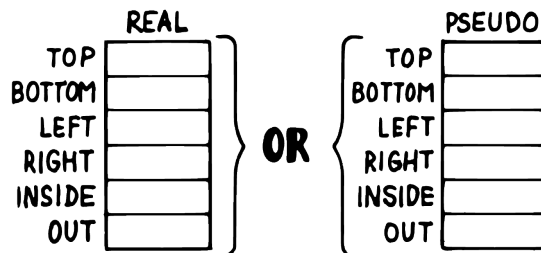


Figure 12: Two labeled columns of boxes, *REAL* and *PSEUDO*, each holding the same field names, with the word *OR* between them – either column can be the one currently in use.

active-profile always holds a map — just not always the same one. Reading or writing through it never needs to know or ask which profile is currently selected; **use-comfort** and **use-eco** are the only two words in the whole program that do:

```
use-eco
active-profile @ "heat-to" m:@ . cr drop    \ => 62

active-profile @ "heat-to" 60 m:! drop

use-comfort
active-profile @ "heat-to" m:@ . cr drop    \ => 68

use-eco
active-profile @ "heat-to" m:@ . cr drop    \ => 60
```

```
62
68
60
```

This is [code/ch08/profiles.8th](#), run and verified. Nudging the eco profile's setpoint down to 60 has no effect on comfort's 68 — confirmed by switching to comfort and back — and switching *back* to eco recovers exactly the nudged value, not the original 62, because **eco** itself was the thing edited, not a copy of it. This is a genuinely different move from the draft/commit pattern above: there, editing happened on a disposable clone, and only a deliberate **commit-resize** folded it back in. Here, both states are the real thing, all the time, and switching between them is nothing more than changing which one **active-profile** currently points at — the same trick Chapter 1's **apples** used to swap an entire variable's identity, now applied to a whole bundle of values instead of one.

Redirecting a Word's Behavior

Brodie's chapter closes with a technique he invented himself and named **DOER/MAKE**: a way to declare a word whose *behavior* — not just its data — can be swapped out after the fact. The mechanism is genuinely clever and genuinely awkward to explain: **MAKE somename** doesn't run code, it *compiles* everything after it, up to the next **;**, as **somename**'s new definition, and then silently ends whatever word it appeared inside — meaning a definition that calls **MAKE** twice is secretly writing the bodies of two *other* words, not two steps of its own. It works, but it demands holding that compile-time sleight of hand in mind every time you read it.

8th has a direct equivalent that doesn't ask for that: **defer:** and **w:is**. Chapter 3 already used **defer:** once, for a forward reference — naming a word before its behavior exists yet, so a foundational piece of code can call something a later component fills in. That's one of **defer:**'s two jobs. The other is exactly Brodie's **DOER/MAKE** motivation: letting a word's actual behavior change at runtime, deliberately, more than once.

```
defer: emit-log \ s --

: emit-log-console \ s --
  . cr ;

' emit-log-console w:is emit-log

: log \ s --
  emit-log ;

"hello" log
```

```
hello
```

`w:is` takes a word reference — the same tick from Chapter 4’s `roman` — and assigns it as the deferred word’s current behavior. Nothing about `log` mentions `emit-log-console` directly; `log` only ever calls `emit-log`, so redirecting `emit-log` redirects every word built on top of `log` too, without touching any of them:

```
"" var, captured

: emit-log-capture \ s --
  captured @ swap s:+ captured ! ;

' emit-log-capture w:is emit-log

"world" log
captured @ . cr    \ => world
```

```
world
```

This is [code/ch08/redirect-log.8th](#), run and verified: `"hello"` prints to the console under the first assignment, and `"world"` prints nothing at all under the second — it’s been captured into a string instead, exactly as `docs/md`’s own example for `defer:` describes doing with 8th’s built-in output words.

A second genuine use for this same mechanism is factoring a single differing step out of an otherwise identical loop — Brodie’s own example was two memory-dump words differing only in whether each unit printed was a byte or a full cell:

```
defer: show-item \ n --

: show-plain \ n --
  . cr ;

: show-tagged \ n --
  "item: %d\n" s:strfmt . ;

: dump-list \ arr --
  ( swap drop show-item ) a:each drop ;
```

```
' show-plain w:is show-item
[ 1 , 2 , 3 ] dump-list

' show-tagged w:is show-item
[ 1 , 2 , 3 ] dump-list
```

```
1
2
3
item: 1
item: 2
item: 3
```

This is [code/ch08/vectored-loop.8th](#), run and verified. `dump-list`'s loop body is written once; `show-item` is the one differing step, reassigned between calls rather than duplicated into two nearly-identical versions of `dump-list` itself — the same factoring instinct Chapter 6 named, aimed here at a step *inside* a loop rather than a whole word.

One corner of Brodie's **DOER/MAKE** material doesn't carry over at all, and it's worth saying so rather than forcing a translation: he uses the same mechanism to implement direct recursion, forward- declaring a word's own name so it can call itself before its definition is finished. 8th doesn't need that trick, or **defer:**, for this — a word can call **recurse** to invoke itself directly, with no forward declaration required:

```
: fact \ n -- n!
  dup 1 n:> if
    dup n:1- recurse n:*
  else
    drop 1
  then ;

5 fact . cr \ => 120
```

```
120
```

This is [code/ch08/recurse.8th](#), run and verified. Where Brodie needed one general-purpose mechanism to solve two different problems — changeable behavior *and* self-reference — 8th simply has two, each aimed at exactly one job.

Summary

Brodie’s “Handling Data” chapter ends with two problems that both come down to giving something more identity than a single stack value or variable can hold. A **map** bundles related values under one name, directly, and **G:clone** gives Chapter 7’s save-and-restore instinct a genuine independent copy to work with instead of a second reference to the same data. Switching between two permanently live states is just reassigning which map a variable currently points at — Chapter 1’s **apples** trick, scaled up from one value to a whole bundle. And where Brodie had to invent **DOER/MAKE** to let a word’s *behavior* change at runtime, 8th already has **defer:** and **w:is** — the same forward- reference tool Chapter 3 used for “not written yet,” reused here for “written, but reassignable,” with **recurse** handling the one job **DOER/MAKE** also did that neither of those two is actually for.

Chapter 9: Minimizing Control Structures

A program built entirely out of small, well-named words needs surprisingly few ``if`s`. Most of the branching a beginner reaches for turns out to be standing in for something else — a decision the dictionary could make by itself, a value that could be calculated instead of chosen, or a shape of control flow that already has its own dedicated word. This chapter is a tour of those substitutes: not “avoid **if** on principle,” but “notice what a conditional is really doing, and reach for the tool built for that job instead.”

The cost of not doing this is cumulative and easy to underestimate. A single extra flag, checked in one new place, looks harmless in isolation. A program that has accumulated a dozen such flags over a few years of maintenance is a program nobody can safely change anymore — every “if this, else if this” chain someone once added “just for this one case” is still there, still being checked, long after anyone remembers why.

Guards and Dispatch, Not Nesting

Picture a checkout terminal: verify the card, verify the balance, then act on whichever button the customer pressed. Written as one deeply nested chain of **if/else**, this reads roughly like: *if the card is valid, then if the balance is sufficient, then if the button was “withdraw,” dispense cash, else if it was “deposit,” accept the envelope, else reject — else (balance not sufficient) reject — else (card not valid) reject*. Untangling which **else** answers which **if** gets harder every time one more case is added, and the two genuinely different kinds of thing this word is doing — *rejecting early for a reason that has nothing to do with which button was pressed* and *deciding what a valid button press should do* — end up tangled together in one nest.

Chapter 4 already introduced half of the fix: **;then** exits a word immediately when its condition is true, so a check that should stop everything doesn’t need an **else** at all — it just leaves. Stack two of those in a row and both rejections are handled before anything about buttons even comes up.

The other half is a genuinely new word: **a:when**. Give it an array of (*test*, *action*) pairs, and it runs each test in order, stopping at the first one that’s true and running the matching action — with one extra convenience: a final, unpaired word at the end of the array acts as the default, run only if nothing else matched.

```
true var, card-valid?
true var, balance-ok?
"" var, pressed

: reject      "transaction rejected\n" . ;
: withdraw    "dispensing cash\n" . ;
: deposit     "accepting envelope\n" . ;

: withdraw?   pressed @ "withdraw" s:= ;
: deposit?    pressed @ "deposit" s:= ;

: checkout \ button --
  card-valid? @ not if drop reject ;then
  balance-ok? @ not if drop reject ;then
  pressed !
  [ ' withdraw? , ' withdraw ,
    ' deposit? , ' deposit ,
    ' reject ]
```

```
a:when ;

"withdraw" checkout

false card-valid? !
"withdraw" checkout

true card-valid? !
"deposit" checkout

"anything-else" checkout
```

```
dispensing cash
transaction rejected
accepting envelope
transaction rejected
```

This is [code/ch09/checkout.8th](#), run and verified. Read `checkout` top to bottom and it says exactly what it does: reject if the card's no good, reject if the balance is short, otherwise act on the button — withdraw, deposit, or (falling all the way through, the way `"anything-else"` does above) reject again, safely, by default. No `else` was needed anywhere, and no case had to be checked once its earlier sibling had already ruled it out.

Combining Conditions, and When Not To

Two related conditions are often more readable combined into one test than nested into two:

```
true var, phone-ringing?
false var, alarm-ringing?

: should-get-up? \ -- flag
  phone-ringing? @ alarm-ringing? @ or ;

should-get-up? . cr \ => true
```

```
true
```

This is [code/ch09/combine.8th](#), run and verified — **or** reads as plainly as the English sentence it's replacing. One thing worth knowing before leaning on this: in a stack language, **or** and **and** can't skip evaluating either side the way **||**/**&&** do in some other languages — both booleans have to already be sitting on the stack, computed, before the combinator can look at either of them. If one of the two checks is expensive and the other is cheap and usually enough on its own, put the cheap one first and nest with **if** instead of combining with **or**, so the expensive check only runs when it's actually needed.

Two Independent Things at Once

Chapters 2, 6, 7, and 8 have all leaned on **caseof** to replace a chain of comparisons with a single lookup — but every one of those tables was indexed by one thing. Some decisions genuinely depend on *two* independent things at once: what key was pressed, say, *and* what mode the program is currently in. A map keyed by a compound string handles this directly, no second dimension of raw memory required:

```
"normal" var, mode

: cursor-left      "cursor left\n" . ;
: cursor-right     "cursor right\n" . ;
: insert-on        "insert mode on\n" . ;
: insert-off       "insert mode off\n" . ;

{
  "normal:left" : ' cursor-left ,
  "normal:right" : ' cursor-right ,
  "normal:i" : ' insert-on ,
  "insert:left" : ' insert-off ,
  "insert:right" : ' insert-off ,
  "insert:i" : ' insert-off
} constant actions

: dispatch \ key --
  mode @ ":" s:+ swap s:+ actions swap caseof ;

"i" dispatch
"insert" mode !
"left" dispatch
"i" dispatch
```

```
insert mode on
insert mode off
insert mode off
```

This is [code/ch09/mode-dispatch.8th](#), run and verified. "normal" ":" "i" becomes the single key "normal:i" before ever touching **actions**, so the two-dimensional problem — which key, in which mode — collapses back into the same one-dimensional lookup **caseof** already knows how to do. Adding a third mode, or a key that behaves the same way in every mode, costs new table rows, not new code.

What You’ve Already Learned, Revisited

A surprising amount of “minimizing control structures” is really “remember the tools from earlier chapters, and reach for them here too”:

- **One word per situation.** `n:+` and `s:+` are different words rather than one smart `+` that inspects its arguments and decides — Chapter 1’s namespace argument, restated: let the *dictionary* make the decision, rather than writing code that makes it every time.
- **Calculate rather than decide**, wherever the two outcomes of a decision are really just two numbers — Chapter 4’s whole “Calculation, Data Structure, or Logic” section.
- **A repeated decision belongs in a table**, whether that table holds plain data (Chapter 2’s parking rates) or word references to invoke (this chapter’s **actions**, Chapter 6’s **channel-actions**).
- **A flag that changes a whole group of things at once usually wants vectoring, not more ``if`s`** — Chapter 7’s **quiet-depth** counter and Chapter 8’s **defer:/w:is** are both this same move: change one thing “at the bottom,” and every caller built on top of it inherits the change for free, with no **if** added anywhere else.
- **Don’t test for something that can’t possibly happen.** A word that trusts its caller to respect its own stack-effect comment doesn’t need to re-check what the caller already promised — a guard belongs at the boundary where genuinely untrusted input enters a program, not copied into every internal word downstream of that boundary.

Calculating Instead of Deciding

Two small, concrete cases are worth their own look, because the “calculate instead of decide” instinct sometimes needs a nudge to spot.

Clamping a value to a floor or ceiling is a decision only if you write it as one:

```
0 var, value
value @ 1 n:- 0 n:max value !
value @ . cr    \ => 0
```

```
0
```

This is [code/ch09/clip.8th](#), run and verified. `n:max` folds “don’t let it go below zero” into the arithmetic itself — there’s no `if` to read, because there isn’t one.

A subtler version of the same idea: packing several independent on/off values into one number. Suppose six independent pixels, each on or off, need to become a single combined code — the classic shape behind everything from status registers to old dot-matrix character sets. Precompute each pixel’s weight as a power of two, and the whole thing becomes a sum, not a chain of tests:

```
[ 0 , 1 , 0 , 1 , 1 , 0 ] var, pixels
[ 1 , 2 , 4 , 8 , 16 , 32 ] constant weights

locals:
: contribution \ i -- n
  "i" w:!
  weights "i" w:@ caseof
  pixels @ "i" w:@ caseof
  n:* ;

: character \ -- code
  160
  0 contribution n:+
  1 contribution n:+
  2 contribution n:+
  3 contribution n:+
  4 contribution n:+
  5 contribution n:+ ;
```

```
character . cr    \ => 186
```

186

This is [code/ch09/pixel-code.8th](#), run and verified. `contribution` adds a pixel’s weight only if that pixel is on (`0 n:*` erases an off pixel’s contribution automatically) — no `if` decides whether to add each weight, because multiplying by 0 or 1 already *is* that decision, expressed arithmetically instead of as a branch.

Store the Value, Not a Flag About It

A flag that exists only so some later code can pick between two numbers is often a sign the number itself should have been stored instead. Suppose a color can optionally be marked “light” — brighter, by one extra bit:

```
0 constant black
1 constant blue
2 constant green

: light \ color -- color
  8 n:bor ;

blue light . cr    \ => 9
```

9

This is [code/ch09/light.8th](#), run and verified. `light` doesn’t set a flag that something else has to check later — it directly produces the *actual color value* with the brightness bit already folded in, ready to use exactly like any other color from that point on. Nothing downstream needs to know a bit was ever set conditionally; there’s no flag to forget to reset.

`n:bor` is worth pausing on: it’s a genuinely different word from the `or` used earlier in this chapter, not just a fussy naming detail. `or` combines two *booleans*; `n:bor` combines the *bits* of two *numbers*. Classic Forth could blur this distinction because its `true` was represented as an actual integer (all bits set) and its `AND/OR` did double duty as both logic and bit manipulation — the basis for a once-common trick of using a boolean directly as a number to eliminate an `IF`. That trick doesn’t

exist in 8th, for a clean reason: 8th’s booleans are their own real type, not numbers wearing a costume, so a boolean can’t be handed to **n:bor** or added to anything at all. Where Forth had one overloaded tool, 8th has two honest ones — worth the one extra namespace prefix.

A Trick That Doesn’t Travel

Classic Forth has a technique for exiting a word early from *inside* a loop or a deeply nested conditional, by directly popping a return address off the return stack — **R> DROP**, used carefully, jumps straight to the calling word’s next instruction, skipping everything still queued up in between. It’s a real technique, and a genuinely dangerous one even in Forth: get the bookkeeping wrong and you corrupt the very mechanism the language uses to know where to resume.

This one doesn’t have an 8th equivalent, and — per Chapter 7 — it doesn’t need one. 8th’s **>r/r>** are deliberately *not* the real return stack; there is no return address sitting there to pop early in the first place. The two problems this trick solved in Forth already have safe, dedicated 8th answers shown earlier in this book: exiting a single word early is **;then** (Chapter 4), and stopping a search loop the moment a match is found is **break** (used the same way **a:each** already iterates a container). Neither one touches anything the language depends on to find its way home.

Summary

Most conditionals worth removing aren’t really about **if** at all — they’re a decision standing in for a lookup (**caseof**, or now **a:when** for a sequence of tests with a built-in default and **caseof** again for two independent dimensions at once), a calculation written out longhand (**n:max** for clipping, arithmetic weights for packing independent bits), or a flag remembering something that could have been stored as the real value or the real behavior in the first place (Chapter 7 and 8’s **vectoring**). None of this is about eliminating **if** on principle — a program with zero conditionals isn’t the goal, one where every remaining conditional is actually earning its place is. And where Forth needed a genuinely risky return-stack trick to get a safe early exit, 8th simply already has one.

Epilogue: 8th's Effect on Thinking

A book about a language usually ends when the last example runs. This one has one chapter's worth of technique left unlearned by design: nothing in the preceding nine chapters was ever really about `if`, or `caseof`, or the stack. Those were the vocabulary. What they were vocabulary *for* is worth naming directly, now that the vocabulary is in place.

Naming outlives the language you learned it in. Chapter 1 opened with a claim that's easy to read past on a first pass: that 8th's namespaces formalize something Forth programmers had to do by discipline, and that most languages since have had to reinvent it under some other name — packages, modules, headers. The specific mechanism changes every decade. The underlying question doesn't: *what is this thing called, from outside the place that built it?* `mode@` and `set-mode` answer that question for a thermostat's mode; `checkout`, `withdraw?`, and `deposit` answer it for a transaction; `apples` answered it for a tally that turned out to need two colors. None of those three examples share a line of code. They share a habit — ask what a caller needs to know, build exactly that much vocabulary, and hide the rest — and that habit doesn't expire when you close this book and open a language with braces instead of a stack.

Small enough to hold in your head, on purpose. Several things in this book turned out to be honest gaps rather than missing features: 8th performs no optimization beyond tail calls, because an optimizer that silently rewrites your program is a mechanism you can no longer fully trust. Its `>r/r<` aren't the real return stack, because a shortcut that can corrupt how the language finds its way home isn't a shortcut worth having. Its booleans are a real type, not an integer in a costume, because a value that's sometimes a number and sometimes a truth depending on context is a value you have to think about twice every time you see it. None of these are limitations you route around. They're the language declining to be clever on your behalf, so that the model in your head and the model actually running stay the same size. A program built this way doesn't scale by getting more mechanisms for you to remember — it scales by staying made of the same few honest pieces, more of them, arranged more ways. That's what “human-scale” means in practice: not that the problems stay small, but that the units you're reasoning in never have to grow past what a person can actually hold.

Factoring is a way of noticing, not just a way of writing. Chapter 6 gave factoring a checklist — repeated code, a comment that wants to be a name, a decision that keeps showing up in slightly different clothes. The checklist is really describing a kind of attention: the moment you catch yourself writing the same shape twice, whether that shape is six lines of 8th or two paragraphs of an email you’ve sent before. Most people develop that attention eventually, on their own, from enough repeated frustration. Writing in a language where the reward for noticing is immediate — factor it out, watch three call sites shrink to one — teaches it faster, and teaches you to trust the instinct once it’s there. **channel-actions, parking-fee, roman’s** two tables: none of them were factored because a rule said to. They were factored because, once written out plainly, the repetition was impossible not to see.

Knowing what *not* to build is its own skill, separate from knowing how to build. This book spent real time on things 8th deliberately doesn’t offer: no general metaprogramming to replace with Forth’s **CREATE/DOES>**, no object system reached for out of habit, no decision table built two dimensions bigger than the problem actually has. Every one of those absences turned out to be a place where a simpler, more direct tool — a native array, a plain namespace, a compound string key — was already sitting there once the urge to generalize was set aside. Chapter 2 said it about data structures specifically: size the solution to the problem you have, not the one you can imagine, because the parts you can’t predict are exactly the parts you’ll want to have hidden behind a word, ready to change, not already locked into a general-purpose shape you’ll have to work around later. That’s a harder discipline than it sounds, because building the general version usually *feels* like the responsible choice, right up until you’re maintaining it.

What travels. Say you close this book and spend the next year writing in a language with none of 8th’s stack, none of its namespaces, none of its interactive, edit-and-watch-it-run rhythm. Some of what you practiced here comes along anyway, because it was never really about the stack. You’ll still ask what a piece of code should be called from the outside before you decide how it works on the inside. You’ll still notice the second time you write something you wrote last week, and reach for a name instead of a copy. You’ll still catch yourself reaching for a general mechanism and ask, honestly, whether the problem in front of you actually needs one, or whether a smaller, plainer answer was sitting there the whole time. A concatenative, stack-based language makes those habits unusually visible while you’re learning them, the way learning to draw makes you notice shadows you’d

walked past your whole life. The shadows were always there. Now you can't stop seeing them.